

ĐẠI HỌC KINH TẾ QUỐC DÂN
TRƯỜNG CÔNG NGHỆ - KHOA CÔNG NGHỆ THÔNG TIN



**BÁO CÁO BÀI TẬP LỚN MÔN KIỂM THỬ
VÀ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM**

Đề tài:

Kiểm thử tự động Net Selenium
Hệ thống quản lý và kết nối CLB - Sinh viên NEU

Giảng viên hướng dẫn : TS. Lê Thị Hoài Thu

Nhóm sinh viên thực hiện : 01

Lớp tín chỉ : CNTT1178(225)_01

Hà Nội – 03/2025

DANH SÁCH THÀNH VIÊN VÀ PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	Mã sinh viên	Lớp chuyên ngành	Vai trò	Phân công công việc	Đánh giá mức độ hoàn thành
1	Vũ Thị Vân Anh	11210832	Kinh tế Phát triển 63B	Thành viên	Làm nội dung, thuyết trình	100%
2	Nguyễn Thị Thu Hà	11235995	Khoa học máy tính 65	Thành viên	Viết báo cáo, làm slide	100%
3	Bùi Thị Anh Thư	11236033	Khoa học máy tính 65	Thành viên	Code web, viết test case, code test	100%
4	Trần Hoài Thu	11236032	Khoa học máy tính 65	Nhóm trưởng	Code web, viết test case, code test, thuyết trình	100%
5	Nguyễn Ngọc Huyền Trang	11236037	Khoa học máy tính 65	Thành viên	Làm nội dung, thuyết trình	100%

MỤC LỤC

LỜI MỞ ĐẦU	4
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI	5
1. Bối cảnh hiện tại	5
2. Tầm quan trọng của đề tài	5
3. Mục tiêu nghiên cứu	6
CHƯƠNG 2. CÔNG CỤ KIỂM THỬ SELENIUM	6
1. Giới thiệu về công cụ kiểm thử Selenium	7
1.1. Khái quát về kiểm thử tự động trong phát triển phần mềm	7
1.2. Khái niệm Selenium	9
1.3. Các thành phần trong bộ công cụ Selenium	9
1.4. Cách chọn công cụ Selenium phù hợp	10
1.5. Vai trò của Selenium WebDriver trong đề tài	11
1.6. Ưu nhược điểm của bộ công cụ Selenium WebDriver	12
1.7. So sánh Selenium với các driver kiểm thử ứng dụng Web khác	13
2. Đặc điểm của Selenium	14
3. Cách cài đặt Selenium	15
3.1. Yêu cầu môi trường cài đặt	15
3.2. Cài đặt Visual Studio	16
3.3. Tạo dự án kiểm thử trong Visual Studio	16
3.4. Cài đặt thư viện Selenium thông qua NuGet	17
3.5. Tải và cấu hình browser driver	17
3.6. Tích hợp framework kiểm thử	18
3.7. Kiểm tra môi trường sau cài đặt	19
CHƯƠNG 3: KIẾN TRÚC VÀ CÁC CHỨC NĂNG CHÍNH CỦA HỆ THỐNG	20
1. Kiến trúc tổng quan hệ thống	20
2. Chức năng chính của Hệ thống Quản lý - kết nối CLB – sinh viên NEU	21
CHƯƠNG 4: KẾ HOẠCH TEST	23
1. Phạm vi kiểm thử	23
2. Kịch bản kiểm thử	24
2.1. Đăng ký tài khoản (Sinh viên)	24
2.2. Đăng nhập tài khoản (Sinh viên)	29
2.3. Đăng nhập tài khoản (CLB) (giống Sinh viên)	33
2.4. Đăng ký thành lập CLB (Sinh viên)	33
2.5. Đăng ký tham gia CLB (Sinh viên)	38
2.6. Phê duyệt đơn đăng ký tham gia CLB Sinh viên (CLB)	46

2.7. Phê duyệt đơn thành lập CLB	48
3. Báo cáo kết quả kiểm thử	52
KẾT LUẬN	53
DANH SÁCH TÀI LIỆU THAM KHẢO	54

LỜI MỞ ĐẦU

Trong thời đại công nghệ số phát triển mạnh mẽ, các hệ thống thông tin ngày càng đóng vai trò quan trọng trong việc hỗ trợ quản lý và kết nối trong môi trường giáo dục đại học. Đặc biệt, các hoạt động ngoại khóa như câu lạc bộ (CLB) là một phần không thể thiếu trong đời sống sinh viên, góp phần phát triển kỹ năng mềm, mở rộng mối quan hệ và nâng cao trải nghiệm học tập.

Để đáp ứng nhu cầu này, các hệ thống quản lý và kết nối giữa sinh viên và CLB đã được xây dựng nhằm số hóa quy trình đăng ký, tổ chức sự kiện và quản lý hoạt động. Tuy nhiên, việc đảm bảo chất lượng cho các hệ thống này là một thách thức lớn, đặc biệt khi số lượng người dùng tăng cao và các chức năng ngày càng phức tạp.

Trong bối cảnh đó, kiểm thử phần mềm đóng vai trò quan trọng trong việc phát hiện lỗi, đảm bảo tính chính xác và nâng cao độ tin cậy của hệ thống. Bên cạnh kiểm thử thủ công, kiểm thử tự động đang trở thành xu hướng tất yếu nhờ khả năng tiết kiệm thời gian và tăng độ chính xác. Trong đó, công cụ Selenium WebDriver trên nền tảng Microsoft .NET là một giải pháp hiệu quả cho việc kiểm thử giao diện web.

Bài báo cáo này tập trung vào việc nghiên cứu và áp dụng kiểm thử tự động bằng công cụ Selenium cho hệ thống quản lý và kết nối CLB – sinh viên NEU, nhằm đánh giá hiệu quả của phương pháp này trong việc đảm bảo chất lượng phần mềm.

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1. Bối cảnh hiện tại

Trong những năm gần đây, quá trình chuyển đổi số trong lĩnh vực giáo dục đã thúc đẩy việc phát triển các hệ thống quản lý trực tuyến nhằm nâng cao hiệu quả vận hành và trải nghiệm người dùng. Tại các trường đại học, đặc biệt là các trường có quy mô lớn như Đại học Kinh tế Quốc dân (NEU), số lượng câu lạc bộ và sinh viên tham gia hoạt động ngoại khóa ngày càng gia tăng.

Tuy nhiên, việc quản lý các hoạt động này bằng phương pháp thủ công hoặc các công cụ rời rạc thường gặp nhiều hạn chế như:

- Khó kiểm soát thông tin thành viên
- Quy trình đăng ký phức tạp, tốn thời gian
- Dễ xảy ra sai sót trong việc xử lý yêu cầu
- Thiếu tính đồng bộ giữa các bộ phận

Do đó, việc xây dựng một hệ thống quản lý và kết nối CLB – sinh viên là cần thiết nhằm tự động hóa các quy trình như đăng ký tham gia CLB, đăng ký sự kiện và xử lý các yêu cầu liên quan.

Tuy nhiên, cùng với sự phát triển của hệ thống, yêu cầu về chất lượng phần mềm cũng ngày càng cao. Các lỗi trong hệ thống có thể ảnh hưởng trực tiếp đến trải nghiệm người dùng và uy tín của tổ chức. Vì vậy, kiểm thử phần mềm, đặc biệt là kiểm thử tự động, trở thành một phần không thể thiếu trong quá trình phát triển hệ thống.

2. Tầm quan trọng của đề tài

Đề tài kiểm thử hệ thống quản lý và kết nối CLB – sinh viên NEU có ý nghĩa quan trọng cả về mặt lý thuyết và thực tiễn.

Về mặt thực tiễn, việc áp dụng kiểm thử tự động giúp: Phát hiện lỗi nhanh chóng và chính xác

- Giảm thiểu chi phí kiểm thử trong dài hạn
- Tăng độ tin cậy và ổn định của hệ thống
- Nâng cao trải nghiệm người dùng

Đặc biệt, với các hệ thống có nhiều chức năng tương tác như đăng ký, phê duyệt giữa nhiều role người dùng việc kiểm thử thủ công sẽ gặp nhiều hạn chế về thời

gian và độ chính xác. Khi đó, việc sử dụng các công cụ kiểm thử tự động như Selenium WebDriver giúp tự động hóa các kịch bản kiểm thử lặp lại, từ đó nâng cao hiệu quả kiểm thử.

Về mặt học thuật, đề tài giúp sinh viên:

- Hiểu rõ quy trình kiểm thử phần mềm
- Tiếp cận với các công cụ kiểm thử hiện đại
- Áp dụng kiến thức lý thuyết vào thực tế
- Phát triển kỹ năng lập trình và tư duy kiểm thử

Ngoài ra, việc sử dụng nền tảng Microsoft .NET còn giúp nâng cao khả năng phát triển phần mềm theo hướng chuyên nghiệp và phù hợp với nhu cầu thực tế của doanh nghiệp.

3. Mục tiêu nghiên cứu

Đề tài hướng tới các mục tiêu chính sau:

- Nghiên cứu tổng quan về kiểm thử phần mềm và kiểm thử tự động
- Tìm hiểu và áp dụng công cụ Selenium WebDriver trong kiểm thử giao diện web
- Xây dựng các kịch bản kiểm thử cho hệ thống quản lý và kết nối CLB – sinh viên NEU
- Thực hiện kiểm thử các chức năng chính của hệ thống, bao gồm:
 - o Đăng ký và đăng nhập tài khoản
 - o Đăng ký thành lập CLB
 - o Đăng ký tham gia CLB
 - o Phê duyệt đơn đăng ký tham gia CLB
 - o Phê duyệt đơn thành lập CLB
- Đánh giá hiệu quả của kiểm thử tự động so với kiểm thử thủ công
- Đề xuất các hướng cải tiến nhằm nâng cao chất lượng hệ thống và quy trình kiểm thử

Thông qua các mục tiêu trên, đề tài không chỉ góp phần đảm bảo chất lượng cho hệ thống mà còn giúp nhóm hiểu rõ hơn về vai trò của kiểm thử trong phát triển phần mềm hiện đại.

CHƯƠNG 2. CÔNG CỤ KIỂM THỬ SELENIUM

1. Giới thiệu về công cụ kiểm thử Selenium

1.1. Khái quát về kiểm thử tự động trong phát triển phần mềm

Trong thực tế, kiểm thử được thực hiện theo hai hướng chính là thủ công và tự động. Kiểm thử thủ công có ưu điểm về tính linh hoạt, nhưng khi hệ thống mở rộng và cần kiểm thử lặp lại nhiều lần, phương pháp này bộc lộ hạn chế về thời gian, độ chính xác và hiệu quả hồi quy. Vì vậy, kiểm thử tự động ngày càng trở thành xu hướng phù hợp trong phát triển phần mềm hiện đại.

Kiểm thử tự động sử dụng các kịch bản có sẵn để mô phỏng hành vi người dùng, từ đó tự động thực hiện các bước kiểm thử và xác minh kết quả. Đối với hệ thống quản lý và kết nối Câu lạc bộ – sinh viên NEU, đây là hướng tiếp cận phù hợp vì giúp tiết kiệm thời gian, nâng cao độ chính xác và tăng hiệu quả kiểm thử lâu dài.

Tiêu chí	Kiểm thử thủ công	Kiểm thử tự động
Định nghĩa	Test case được thực hiện thủ công bởi tester.	Tester cần viết test script và lựa chọn công cụ để tự động hóa việc kiểm thử.
Thời gian xử lý	Cần nhiều thời gian và nhân lực.	Thời gian kiểm thử nhanh hơn so với manual testing.
Exploratory Testing / Kiểm thử khám phá	Có thể thực hiện kiểm thử khám phá.	Không hỗ trợ kiểm thử khám phá.
Thay đổi UI	Những thay đổi nhỏ trên giao diện thường không ảnh hưởng nhiều đến quá trình kiểm thử.	Chỉ cần thay đổi nhỏ trên UI cũng có thể yêu cầu cập nhật lại script.

Độ tin cậy	Kết quả có thể kém ổn định do dễ phát sinh lỗi từ con người.	Kết quả đáng tin cậy hơn vì được thực thi bằng công cụ và script.
Đầu tư	Chủ yếu cần nguồn nhân lực.	Cần đầu tư công cụ và nhân sự có kỹ năng automation.
Báo cáo	Kết quả thường được lưu thủ công trên Excel, Word hoặc tài liệu tương tự.	Kết quả có thể được lưu trên hệ thống để các bên liên quan dễ theo dõi.
Sự quan sát của con người	Cần sự quan sát của con người để đánh giá tính thân thiện với người dùng.	Hầu như không có yếu tố quan sát trực tiếp của con người.
Kiểm thử hiệu năng/ Performance Testing	Không phù hợp để thực hiện kiểm thử hiệu năng.	Có thể thực hiện thông qua các công cụ tự động phù hợp.
Kiến thức lập trình	Không yêu cầu khả năng lập trình.	Cần có kiến thức lập trình để xây dựng test script.
Cách tiếp cận tốt	Phù hợp khi bộ test chỉ cần chạy lại 1 hoặc 2 lần.	Hiệu quả khi cần chạy lại bộ script nhiều lần.
Sử dụng khi nào?	Phù hợp với kiểm thử khám phá, khả năng sử dụng và kiểm thử ad-hoc.	Phù hợp với kiểm thử hồi quy, kiểm thử hiệu năng và các trường hợp lặp lại nhiều lần.

1.2. Khái niệm Selenium

Selenium là bộ công cụ kiểm thử tự động mã nguồn mở dành cho ứng dụng web, hỗ trợ mô phỏng các thao tác của người dùng trên trình duyệt như nhập dữ liệu, nhấn nút, gửi biểu mẫu và kiểm tra kết quả hiển thị. Được phát triển từ năm 2004 bởi ThoughtWorks, Selenium ngày càng hoàn thiện và mở rộng khả năng hỗ trợ nhiều trình duyệt, hệ điều hành và ngôn ngữ lập trình. Hiện nay, Selenium có thể hoạt động trên các trình duyệt phổ biến như Chrome, Firefox, Safari, Edge và trên các nền tảng như Windows, Linux, macOS, qua đó hỗ trợ hiệu quả cho kiểm thử đa trình duyệt và đa nền tảng.

1.3. Các thành phần trong bộ công cụ Selenium

Selenium gồm nhiều thành phần, mỗi thành phần phục vụ một mục đích khác nhau trong kiểm thử tự động ứng dụng web.

a. Selenium IDE

Selenium IDE là tiện ích mở rộng trên trình duyệt, cho phép ghi lại và phát lại các thao tác của người dùng. Công cụ này phù hợp với người mới bắt đầu và các kịch bản kiểm thử đơn giản, nhưng khả năng tùy biến còn hạn chế.

b. Selenium RC

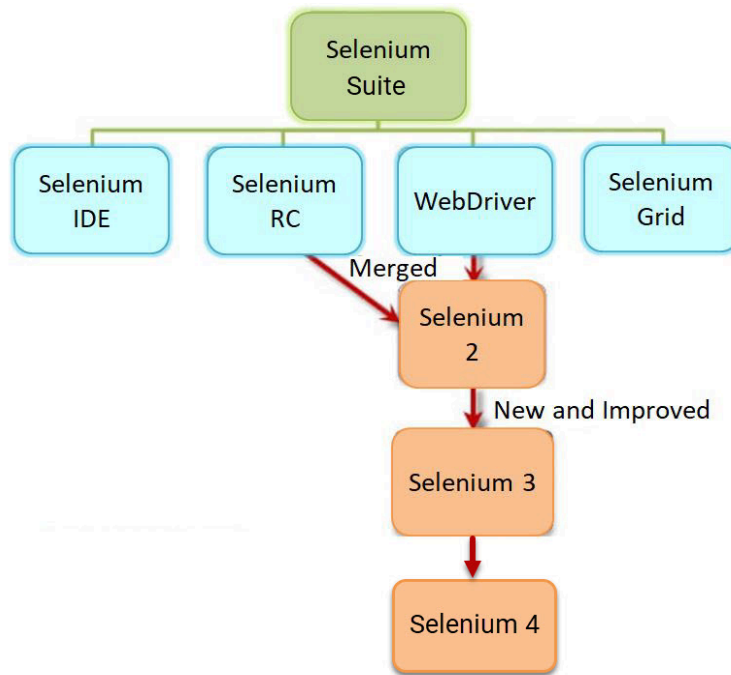
Selenium RC là framework kiểm thử đời đầu của Selenium, cho phép viết kịch bản bằng nhiều ngôn ngữ lập trình. Tuy nhiên, do phụ thuộc vào máy chủ trung gian và JavaScript, công cụ này dần bộc lộ nhiều hạn chế và hiện nay ít được sử dụng.

c. Selenium WebDriver

Selenium WebDriver là thành phần hiện đại và phổ biến nhất của bộ công cụ Selenium. WebDriver cho phép điều khiển trực tiếp trình duyệt bằng mã nguồn, có tính linh hoạt cao và phù hợp với các dự án kiểm thử quy mô lớn. Đây cũng là thành phần được lựa chọn trong đề tài này.

d. Selenium Grid

Selenium Grid hỗ trợ chạy kiểm thử song song trên nhiều trình duyệt và nhiều môi trường khác nhau. Công cụ này giúp rút ngắn thời gian thực thi và hỗ trợ tốt cho kiểm thử đa nền tảng.



Hình 2.1.3. Các thành phần của bộ công cụ Selenium

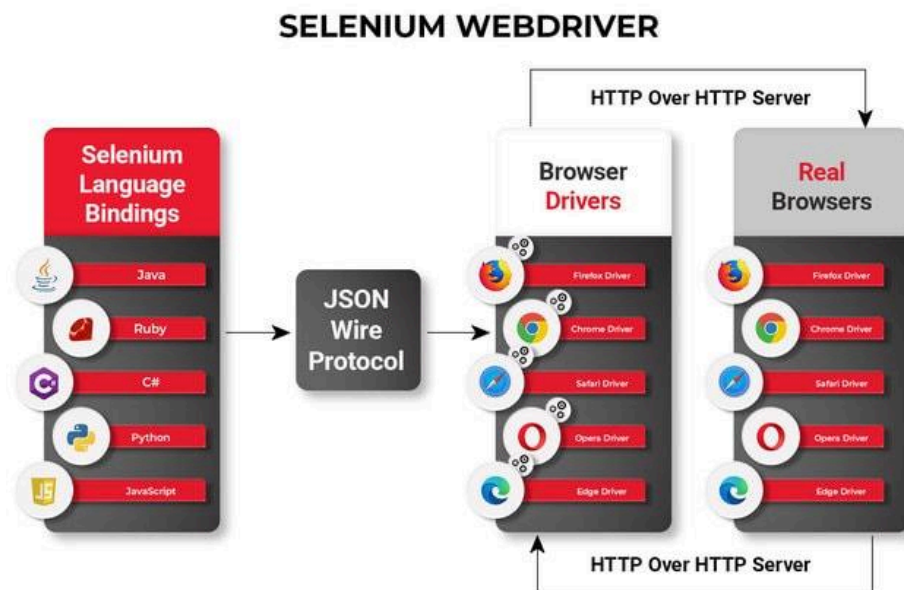
1.4. Cách chọn công cụ Selenium phù hợp

Công cụ	Trường hợp sử dụng phù hợp
Selenium IDE	Để tìm hiểu về các khái niệm về thử nghiệm tự động và Selenium, bao gồm: - Các lệnh Selen như kiểu, mở, bấm và đợi, xác nhận, xác minh, v.v. - Các trình định vị như id, name, xpath, css selector, v.v. - Thực thi mã JavaScript tùy chỉnh bằng cách sử dụng chạy script - Xuất các trường hợp thử nghiệm ở các định dạng khác nhau - Tạo ca kiểm thử với ít hoặc không có kiến thức về lập trình - Để tạo các trường hợp kiểm thử đơn giản và các bộ kiểm thử mà bạn có thể xuất sau này sang RC hoặc WebDriver. - Để kiểm tra một ứng dụng web chỉ chống lại Firefox.

Selenium RC	- Để thiết kế một bài kiểm tra sử dụng một ngôn ngữ hơn Selenese - Để chạy kiểm thử của bạn với các trình duyệt khác nhau (ngoại trừ HtmlUnit) trên các hệ điều hành khác nhau. - Để triển khai các kiểm thử của bạn trên nhiều môi trường sử dụng lưới Selenium. - Để kiểm tra ứng dụng của bạn dựa trên trình duyệt mới hỗ trợ JavaScript. - Để kiểm tra các ứng dụng web với các kịch bản dựa trên AJAX phức tạp.
WebDriver	- Để sử dụng một ngôn ngữ lập trình nhất định trong việc thiết kế trường hợp thử nghiệm của bạn. - Để kiểm tra các ứng dụng có nhiều chức năng dựa trên AJAX. - Để thực hiện các kiểm tra trên trình duyệt HtmlUnit. - Để tạo kết quả kiểm tra tùy chỉnh.
Selenium Grid	- Để chạy các kịch bản lệnh Selenium RC của bạn trong nhiều trình duyệt và hệ điều hành đồng thời. - Để chạy một bộ thử nghiệm khổng lồ, cần phải hoàn thành trong thời gian sớm nhất có thể.

1.5. Vai trò của Selenium WebDriver trong đề tài

Mặc dù Selenium là một bộ công cụ gồm nhiều thành phần, song trong thực tiễn triển khai kiểm thử tự động cho đề tài, Selenium WebDriver là thành phần giữ vai trò trung tâm. WebDriver cho phép xây dựng và thực thi các kịch bản kiểm thử bằng mã nguồn, đồng thời hỗ trợ tương tác trực tiếp với trình duyệt thông qua các driver chuyên biệt như ChromeDriver hoặc GeckoDriver.



Hình 2.1.4. Cấu trúc hoạt động của Selenium WebDriver

Việc lựa chọn Selenium WebDriver phù hợp với đặc thù của hệ thống quản lý và kết nối CLB – sinh viên NEU, bởi đây là ứng dụng web có nhiều chức năng thao tác trực tiếp trên giao diện như đăng ký tài khoản, đăng nhập, gửi đơn ứng tuyển, đăng ký sự kiện, đặt hàng, phê duyệt yêu cầu và quản lý nội dung. Những chức năng này có thể được mô phỏng hiệu quả thông qua việc thao tác với ô nhập liệu, nút bấm, biểu mẫu và các liên kết điều hướng.

1.6. Ưu nhược điểm của bộ công cụ Selenium WebDriver

Ưu điểm	Nhược điểm
• Là mã nguồn mở miễn phí. • Có tính linh hoạt, vì là cả một bộ công cụ được tích hợp trong một framework duy nhất. • Dễ sử dụng, dễ học, không yêu cầu hiểu biết chuyên sâu về công cụ. • Được phát triển chủ yếu bằng JavaScript, test script dựa trên HTML nên thực thi nhanh và dễ dàng hơn	• Hạn chế cho việc kiểm tra hình ảnh như không thể nhận dạng văn bản bên trong hình ảnh. • Cần có kiến thức trình độ cao về lập trình để sử dụng • Gặp khó khăn khi xử lý các trang được tạo động.

• Hỗ trợ hầu hết các trình duyệt phổ biến như Chrome, Firefox, Opera, Safari... • Hỗ trợ nhiều ngôn ngữ lập trình như Java, C#, Python, PHP, Perl, Ruby, ... • Độc lập với nền tảng, nghĩa là có thể triển khai trên nhiều hệ điều hành như Windows, Linux, Macintosh...	• Khó sử dụng khi thử nghiệm các ứng dụng web sử dụng Ajax hoặc ReactJS. • Không thể tương tác với flash hoặc Java applet.
--	---

1.7. So sánh Selenium với các driver kiểm thử ứng dụng Web khác

Công cụ automation testing	Hệ điều hành hỗ trợ	Ngôn ngữ hỗ trợ	Ứng dụng	Yêu cầu kỹ năng lập trình	Chi phí
Selenium	Windows/ Mac/ Linux	Java, Python, C#, PHP, Javascript, Ruby, Perl	Web, mobile (có Appium)	Trình độ cao	Miễn phí
TestComplete Katalon Studio	Windows/ OS/ Linux	VB, Javascript, Jscript, C++, C#, Angular, Delphi, Ruby on Rails, PHP	Web, mobile, desktop	Trình độ cơ bản đến cao	\$4600 – \$9000
UFT	Windows	VBScript	Web, mobile, desktop	Trình độ cơ bản đến cao	\$2500 – \$3500

Watir	Windows	Ruby, Java, .Net	Web	Trình độ cơ bản	Miễn phí
Ranorex	Windows	C#, Python, VB, .Net, Iron	Web, mobile, desktop	Trình độ cơ bản đến cao	N/A

2. Đặc điểm của Selenium

a. Selenium là bộ công cụ kiểm thử tự động mã nguồn mở

Selenium là bộ công cụ kiểm thử tự động mã nguồn mở và miễn phí dành cho ứng dụng web. Đặc điểm này giúp công cụ dễ tiếp cận trong cả môi trường học thuật lẫn doanh nghiệp, đồng thời hỗ trợ tốt cho việc nghiên cứu và triển khai thực tế mà không phát sinh chi phí bản quyền. Ngoài ra, Selenium còn có cộng đồng hỗ trợ lớn, tài liệu phong phú và khả năng mở rộng cao.

b. Selenium là một hệ sinh thái gồm nhiều thành phần

Selenium không phải là một công cụ đơn lẻ mà là một hệ sinh thái gồm nhiều thành phần như Selenium IDE, Selenium RC, Selenium WebDriver và Selenium Grid. Mỗi thành phần đảm nhiệm một vai trò khác nhau, từ ghi và phát lại thao tác đơn giản đến xây dựng kịch bản kiểm thử nâng cao và chạy kiểm thử song song. Điều này cho thấy Selenium là một nền tảng tương đối toàn diện cho kiểm thử ứng dụng web.

c. Hỗ trợ nhiều trình duyệt và nhiều hệ điều hành

Selenium hỗ trợ nhiều trình duyệt phổ biến như Chrome, Firefox, Safari, Edge và hoạt động trên các hệ điều hành như Windows, Linux, macOS. Nhờ đó, công cụ này giúp kiểm thử viên đánh giá tính tương thích của hệ thống trên nhiều môi trường khác nhau, từ đó nâng cao độ tin cậy của phần mềm khi triển khai thực tế.

d. Hỗ trợ nhiều ngôn ngữ lập trình

Selenium có thể kết hợp với nhiều ngôn ngữ lập trình như Java, C#, Python, PHP, Perl và Ruby. Đây là ưu điểm quan trọng vì cho phép lựa chọn ngôn ngữ phù hợp với nền tảng công nghệ đang sử dụng. Trong đề tài này, việc sử dụng C# là phù hợp với môi trường Visual Studio và nền tảng .NET.

e. Cho phép tương tác trực tiếp với trình duyệt

Một đặc điểm nổi bật của Selenium WebDriver là khả năng giao tiếp trực tiếp với trình duyệt thông qua các driver chuyên biệt. So với Selenium RC, cơ chế này giúp quá trình kiểm thử ổn định hơn, nhanh hơn và phản ánh sát hơn thao tác thực tế của người dùng. Đây là yếu tố rất phù hợp với các chức năng kiểm thử trên giao diện web của hệ thống.

f. Phù hợp với kiểm thử các ứng dụng web động

Selenium, đặc biệt là WebDriver, phù hợp với các ứng dụng web có mức độ tương tác cao, sử dụng JavaScript, AJAX và các thành phần giao diện động. Nhờ đó, công cụ này hỗ trợ tốt cho việc kiểm thử các thao tác như nhập liệu, gửi biểu mẫu, chuyển trang và cập nhật trạng thái trên giao diện. Tuy nhiên, việc xử lý các phần tử động vẫn đòi hỏi người kiểm thử có kiến thức kỹ thuật nhất định.

g. Hỗ trợ thực thi kiểm thử song song và mở rộng quy mô

Thông qua Selenium Grid, Selenium có khả năng chạy đồng thời nhiều kịch bản kiểm thử trên nhiều trình duyệt và nhiều môi trường khác nhau. Đặc điểm này giúp tiết kiệm thời gian thực thi và hỗ trợ tốt cho các dự án có quy mô kiểm thử lớn hoặc cần đánh giá tính tương thích đa nền tảng.

h. Dễ tích hợp với framework kiểm thử và mô hình tổ chức mã nguồn

Selenium có thể tích hợp tốt với các framework kiểm thử và mô hình tổ chức mã nguồn. Khi kết hợp với C# trong Visual Studio, công cụ này hỗ trợ xây dựng framework kiểm thử có cấu trúc, dễ quản lý test case và thuận lợi cho việc áp dụng các mô hình như Page Object Model. Điều này giúp mã kiểm thử rõ ràng hơn và dễ bảo trì hơn trong quá trình phát triển lâu dài.

3. Cách cài đặt Selenium

3.1. Yêu cầu môi trường cài đặt

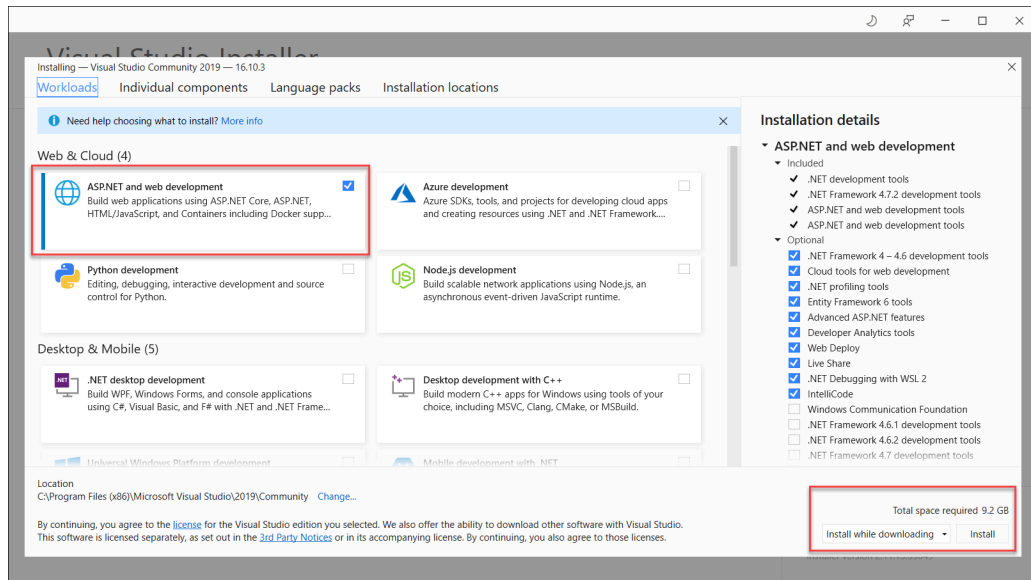
Trước khi cài đặt Selenium, cần chuẩn bị đầy đủ môi trường phát triển. Trước hết là Visual Studio, môi trường phát triển tích hợp của Microsoft, phù hợp cho việc xây dựng và quản lý project kiểm thử trên nền tảng .NET. Bên cạnh đó, hệ thống cần có .NET SDK hoặc framework .NET phù hợp để biên dịch và thực thi mã nguồn C#.

Ngoài môi trường phát triển, máy tính cần được cài đặt ít nhất một trình duyệt như Google Chrome hoặc Mozilla Firefox, kèm theo browser driver tương ứng, chẳng hạn ChromeDriver hoặc GeckoDriver. Đây là thành phần trung gian cho phép Selenium WebDriver giao tiếp trực tiếp với trình duyệt và thực hiện các thao tác kiểm thử tự động.

Trong Visual Studio, các thư viện Selenium thường được cài đặt thông qua NuGet Package Manager. Công cụ này hỗ trợ quản lý package và thư viện phụ thuộc một cách thuận tiện, do đó là thành phần cần thiết trong quá trình thiết lập Selenium với C#.

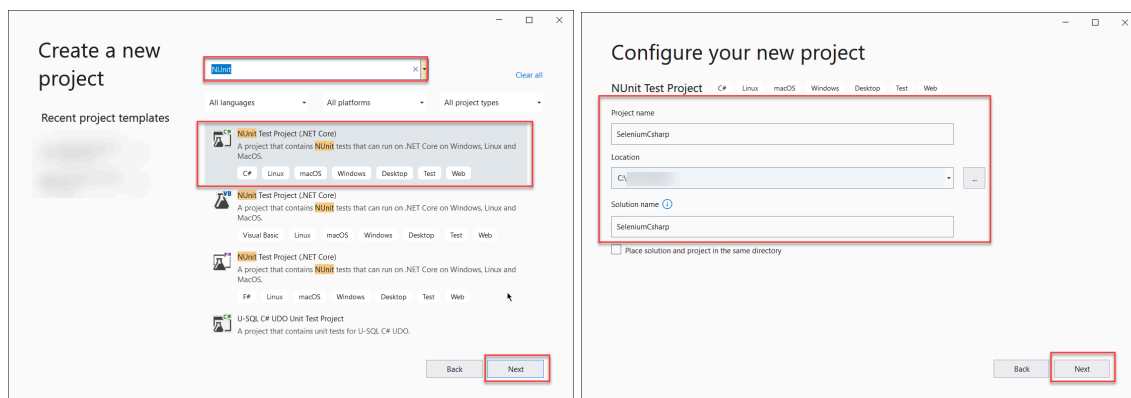
3.2. Cài đặt Visual Studio

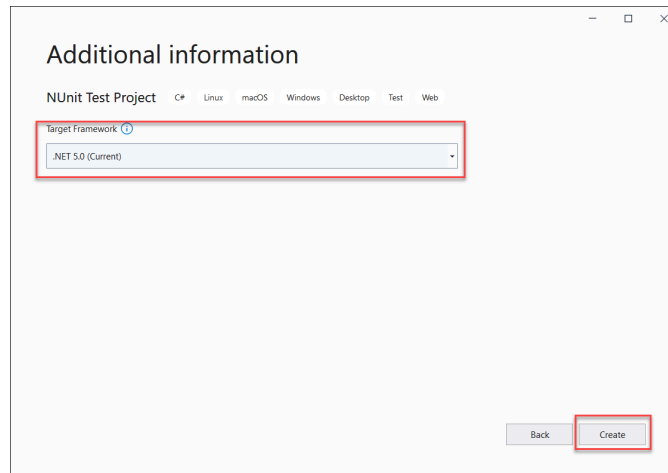
Bước đầu tiên trong quá trình cài đặt là thiết lập môi trường [Visual Studio](#).



3.3. Tạo dự án kiểm thử trong Visual Studio

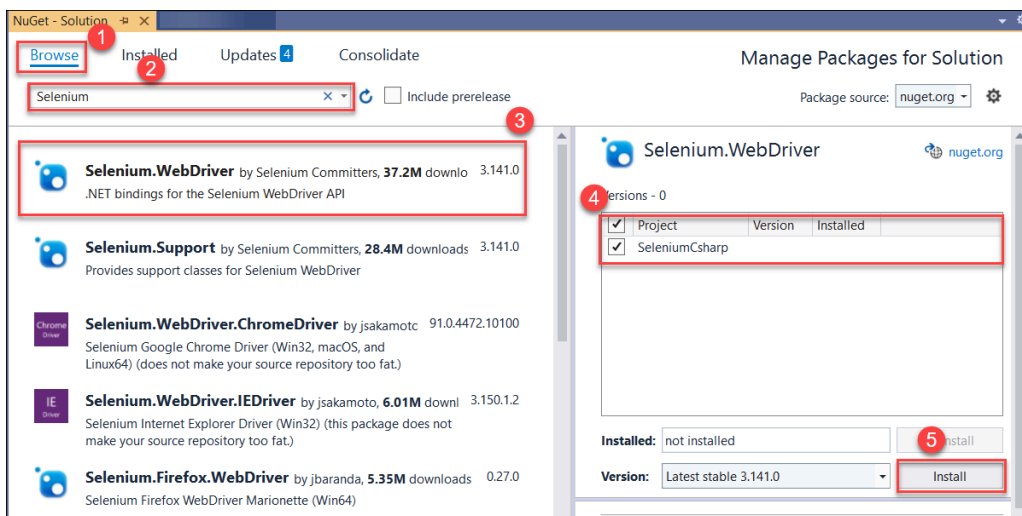
Người dùng chọn chức năng Create a new project trong Visual Studio. Đối với kiểm thử tự động với Selenium bằng C#, loại project được khuyến nghị là NUnit Project trên nền tảng .NET Core. Trong quá trình tạo project, người dùng cần thiết lập tên project, vị trí lưu trữ và tên solution, sau đó lựa chọn phiên bản framework mục tiêu phù hợp.





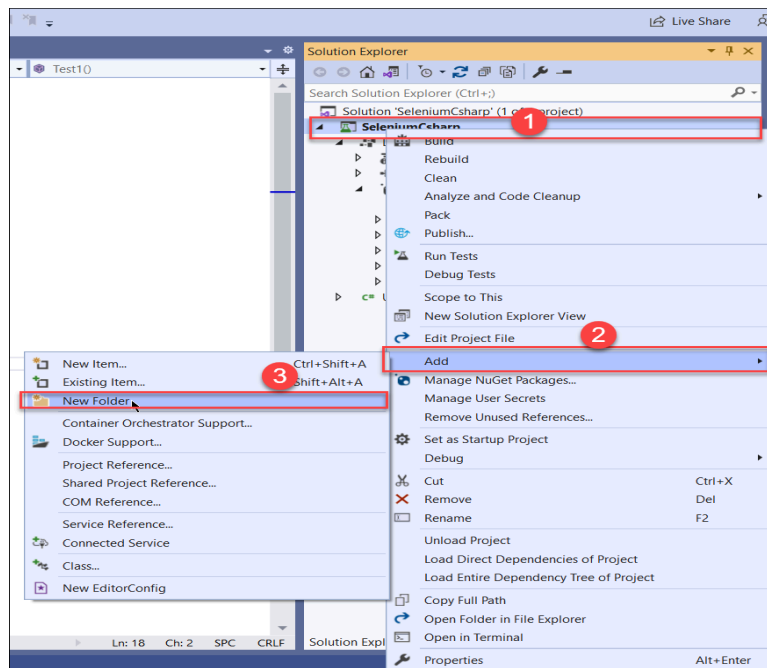
3.4. Cài đặt thư viện Selenium thông qua NuGet

Sau khi đã tạo project kiểm thử, cài đặt các thư viện Selenium cần thiết thông qua NuGet Package Manager. Trong menu của Visual Studio, người dùng truy cập vào Tools → NuGet Package Manager → Manage NuGet Packages for Solution để mở giao diện quản lý package của solution. Tại đây, package đầu tiên cần cài đặt là Selenium WebDriver. người dùng cần cài thêm package Selenium Support cung cấp các thành phần hỗ trợ trong quá trình kiểm thử.



3.5. Tải và cấu hình browser driver

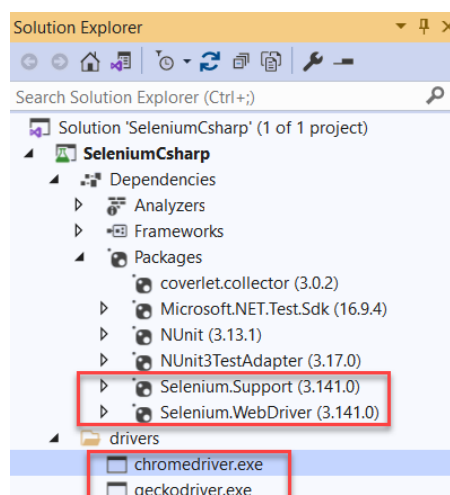
Sau khi cài đặt xong các package Selenium, người dùng nên tạo một thư mục riêng trong project, thường được đặt tên là drivers, để lưu trữ các file thực thi của browser driver. Sau đó, cần tải các driver phù hợp, chẳng hạn như ChromeDriver cho Google Chrome hoặc GeckoDriver cho Mozilla Firefox, rồi sao chép file thực thi vào thư mục này.



SeleniumCsharp > SeleniumCsharp > drivers			
Name	Date modified	Type	Size
chromedriver.exe		Application	10,907 KB
geckodriver.exe		Application	3,407 KB

3.6. Tích hợp framework kiểm thử

Sau khi hoàn tất việc cài đặt thư viện Selenium và browser driver, project cần được tích hợp với một framework kiểm thử phù hợp để tổ chức các test case. Project được tạo theo kiểu NUnit, đây là một framework kiểm thử phổ biến trong môi trường C#. Việc sử dụng NUnit giúp các ca kiểm thử được tổ chức thành các lớp, phương thức và bộ test rõ ràng, đồng thời hỗ trợ quá trình thực thi, đánh giá kết quả và báo cáo lỗi một cách hệ thống.



3.7. Kiểm tra môi trường sau cài đặt

Sau khi hoàn tất cài đặt, cần kiểm tra lại môi trường để bảo đảm project đã sẵn sàng cho việc xây dựng test case. Bước này thường được thực hiện bằng một bài kiểm thử đơn giản có khả năng mở trình duyệt, truy cập một địa chỉ web và kết thúc phiên làm việc. Nếu trình duyệt khởi chạy và thực hiện điều hướng thành công, có thể khẳng định rằng các thành phần như Visual Studio, project C#, Selenium WebDriver, framework kiểm thử và browser driver đã được tích hợp đúng cách.

Việc kiểm tra môi trường sau cài đặt có ý nghĩa quan trọng vì giúp phát hiện sớm các lỗi cấu hình như thiếu package, sai đường dẫn driver, không tương thích phiên bản trình duyệt hoặc lỗi framework. Nếu bỏ qua bước này, các lỗi kỹ thuật có thể bị nhầm lẫn với lỗi của hệ thống cần kiểm thử, từ đó làm giảm độ chính xác của quá trình nghiên cứu.

CHƯƠNG 3: KIẾN TRÚC VÀ CÁC CHỨC NĂNG CHÍNH CỦA HỆ THỐNG

1. Kiến trúc tổng quan hệ thống

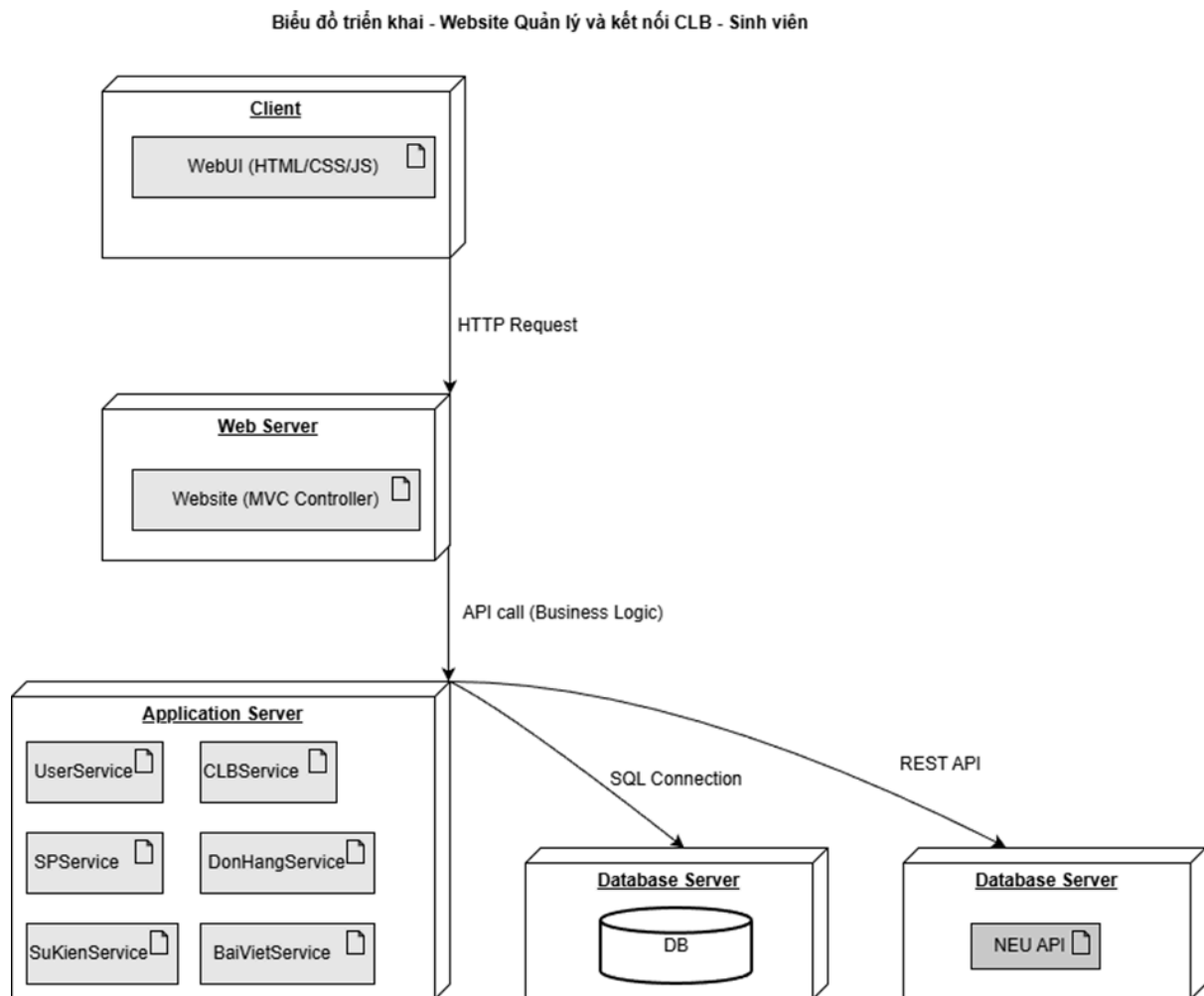
Hệ thống quản lý và kết nối Câu lạc bộ (CLB) – sinh viên NEU là một ứng dụng web được xây dựng nhằm hỗ trợ sinh viên dễ dàng tham gia các hoạt động ngoại khóa, đồng thời giúp các CLB quản lý thành viên, sự kiện và các hoạt động liên quan một cách hiệu quả.

Hệ thống được thiết kế theo mô hình đa tầng (multi-tier architecture), bao gồm các thành phần chính: Client, Web Server, Application Server và Database Server. Các thành phần này tương tác với nhau thông qua các giao thức chuẩn như HTTP và REST API, đảm bảo tính linh hoạt và khả năng mở rộng của hệ thống.

- Client (Trình duyệt người dùng):
Đây là nơi sinh viên (Student và CLB) trực tiếp tương tác với hệ thống thông qua giao diện web được xây dựng bằng các công nghệ như HTML, CSS và JavaScript. Người dùng thực hiện các thao tác như đăng ký, đăng nhập, tham gia CLB, đăng ký sự kiện và đặt hàng tại đây.
- Web Server:
Web Server đóng vai trò tiếp nhận các yêu cầu HTTP từ phía client và xử lý thông qua các bộ điều khiển (Controller) theo mô hình MVC. Thành phần này chịu trách nhiệm điều hướng yêu cầu, xử lý dữ liệu đầu vào và chuyển tiếp các yêu cầu đến tầng xử lý nghiệp vụ.
- Application Server (Tầng xử lý nghiệp vụ):
Đây là thành phần cốt lõi của hệ thống, nơi triển khai các logic nghiệp vụ thông qua các service như: UserService, CLBService, SuKienService, DonHangService, BaiVietService,... Các service này xử lý các chức năng chính như quản lý người dùng, quản lý CLB, xử lý đăng ký sự kiện, đặt hàng và phê duyệt yêu cầu.
- Database Server:
Hệ thống sử dụng cơ sở dữ liệu để lưu trữ toàn bộ thông tin liên quan đến người dùng, CLB, sự kiện, đơn hàng và các dữ liệu khác. Application Server kết nối đến Database Server thông qua các truy vấn SQL để thực hiện việc lưu trữ và truy xuất dữ liệu.

Luồng hoạt động tổng quát của hệ thống diễn ra như sau: người dùng gửi yêu cầu từ trình duyệt (client) đến Web Server thông qua HTTP, sau đó Web Server chuyển yêu cầu đến Application Server để xử lý nghiệp vụ. Kết quả xử lý sẽ được lưu

trữ hoặc truy xuất từ Database Server, sau đó phản hồi lại cho người dùng dưới dạng giao diện web.

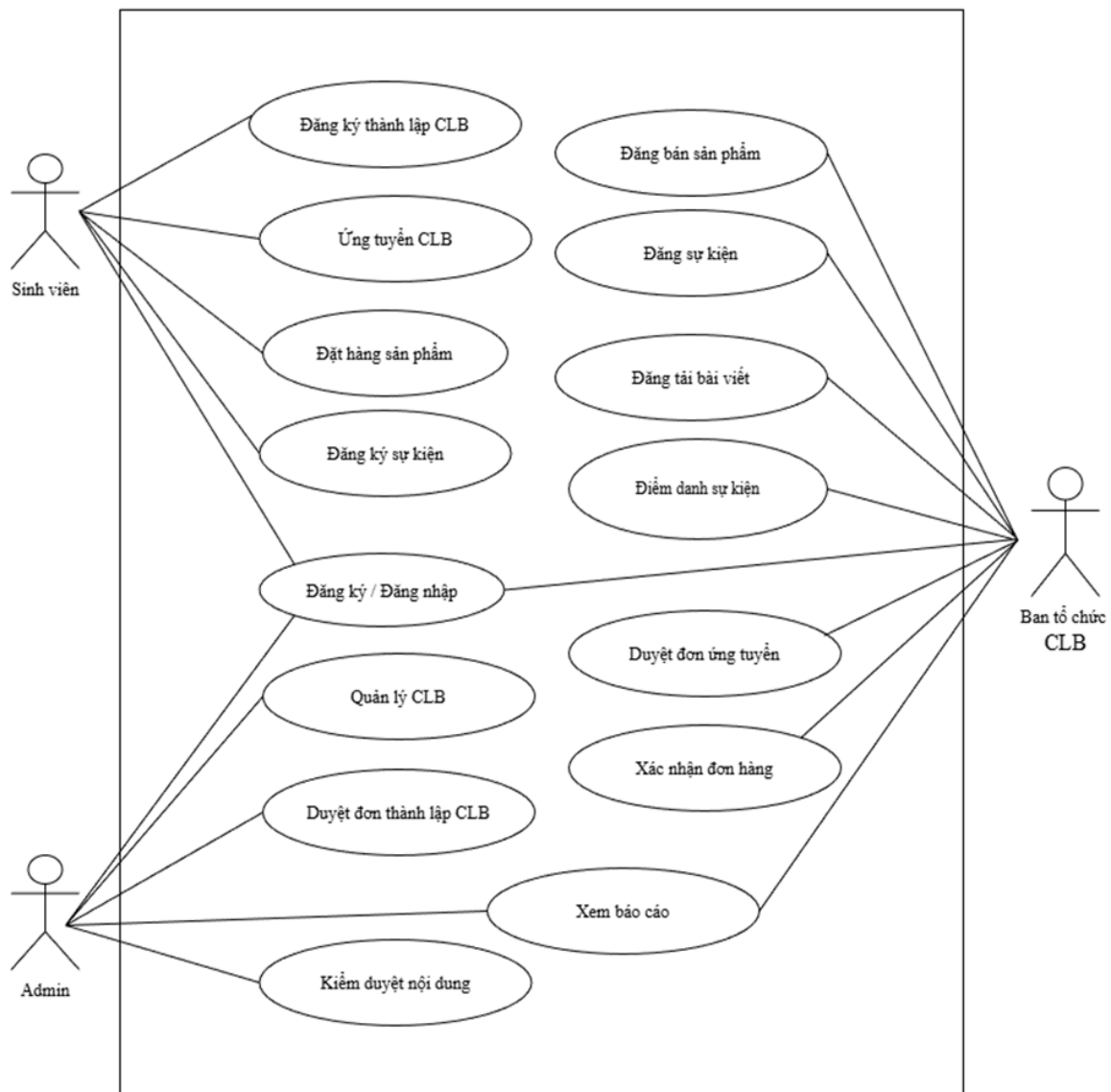


2. Chức năng chính của Hệ thống Quản lý - kết nối CLB – sinh viên NEU

Hệ thống được thiết kế nhằm phục vụ ba nhóm người dùng chính bao gồm: Sinh viên, Ban tổ chức CLB và Quản trị viên. Mỗi nhóm người dùng có các chức năng riêng biệt, phù hợp với vai trò và nhu cầu sử dụng trong hệ thống.

Dưới đây là sơ đồ Use Case tổng quát của hệ thống:

Use case tổng quát Hệ thống Quản lý và kết nối CLB - Sinh viên NEU



CHƯƠNG 4: KẾ HOẠCH TEST

1. Phạm vi kiểm thử

Phạm vi kiểm thử của dự án tập trung vào việc đảm bảo tính đúng đắn về chức năng và quy trình nghiệp vụ của hệ thống quản lý Câu lạc bộ (CLB) trên cả 3 phân quyền người dùng chính với một vài chức năng tiêu biểu:

a. Vai trò: Sinh viên (Student)

Đảm bảo sinh viên có thể thực hiện các tương tác cơ bản và tham gia vào hệ sinh thái CLB.

- Đăng ký & Đăng nhập: Kiểm thử quá trình tạo tài khoản mới và truy cập hệ thống với thông tin hợp lệ/không hợp lệ.
- Đăng ký thành lập CLB: Kiểm thử quy trình gửi form yêu cầu thành lập CLB mới (nhập thông tin, chọn lĩnh vực, mô tả,...).
- Đăng ký tham gia CLB: Kiểm thử chức năng tìm kiếm và gửi đơn xin gia nhập vào các CLB hiện có trên hệ thống.

b. Vai trò: Ban quản lý Câu lạc bộ (Club)

Đảm bảo đại diện CLB có thể quản lý danh sách thành viên và các yêu cầu gia nhập.

- Đăng nhập CLB: Kiểm thử khả năng truy cập vào giao diện quản lý dành riêng cho CLB.
- Phê duyệt đơn tham gia: Kiểm thử luồng nghiệp vụ chấp nhận hoặc từ chối sinh viên đăng ký vào CLB, đảm bảo trạng thái đơn được cập nhật chính xác.

c. Vai trò: Quản trị viên (Admin)

Đảm bảo cấp quản lý cao nhất có thể kiểm soát việc hình thành các tổ chức trong hệ thống.

- Phê duyệt đơn thành lập CLB: Kiểm thử quy trình xét duyệt các yêu cầu mở CLB mới từ phía sinh viên. Đảm bảo khi Admin phê duyệt, CLB đó chính thức xuất hiện trên hệ thống.

Kế hoạch test của dự án này, nhóm không kiểm thử bảo mật chuyên sâu.

2. Kịch bản kiểm thử

2.1. Đăng ký tài khoản (Sinh viên)

a. Kịch bản kiểm thử

STT	Mô tả kịch bản	Các bước kiểm thử	Kết quả mong đợi
1	Kiểm tra giao diện đăng ký	Mở trang đăng ký và kiểm tra các thành phần giao diện	Hiển thị đúng theo thiết kế
2	Đăng ký thành công	1. Nhập đầy đủ thông tin hợp lệ (User, Email (@ neu.edu.vn), Mật , MSV, Họ tên). 2. Nhấn nút đăng ký.	Đăng ký thành công và hệ thống tự động chuyển hướng về trang Đăng nhập.
3	Đề trống tên đăng nhập	1. Đề trống ô Username. 2. Nhập các thông tin còn lại hợp lệ. 3. Nhấn đăng ký.	Hệ thống chặn gửi đơn, trình duyệt báo lỗi "Please fill out this field".
4	Đề trống Email	1. Đề trống ô Email và nhấn đăng ký.	Hệ thống chặn gửi đơn, trình duyệt báo lỗi "Please fill out this field".
5	Đề trống mật khẩu	1. Đề trống ô Password và nhấn đăng ký.	Hệ thống chặn gửi đơn, trình duyệt báo lỗi "Please fill out this field".
6	Đề trống mã sinh viên	1. Đề trống ô Mã sinh viên và nhấn đăng ký.	Hệ thống chặn gửi đơn, trình duyệt báo lỗi "Please fill out this

			field".
7	Đề trống họ và tên	1. Đề trống ô Họ tên và nhấn đăng ký.	Hệ thống chặn gửi đơn, trình duyệt báo lỗi "Please fill out this field".
8	Email sai định dạng	1. Nhập email có đuôi khác (ví dụ: @gmail.com). 2. Nhấn đăng ký.	Hiển thị thông báo lỗi: "Email phải có đuôi @neu.edu.vn".
9	Email thiếu phần định danh	1. Nhập email chỉ có đuôi "@neu.edu.vn". 2. Nhấn đăng ký.	Trình duyệt báo lỗi: "Please enter a part followed by '@'..."
10	Mật khẩu quá ngắn	1. Nhập mật khẩu dưới 6 ký tự. 2. Nhấn Đăng ký.	Hiển thị thông báo lỗi: "Mật khẩu phải có ít nhất 6 ký tự".
11	Trùng tên đăng nhập	1. Nhập Username đã tồn tại. 2. Nhấn Đăng ký.	Hiển thị thông báo lỗi từ server: "Tên đăng nhập này đã tồn tại".
12	Trùng mã sinh viên	1. Nhập MSV đã được đăng ký. 2. Nhấn Đăng ký.	Hiển thị thông báo lỗi từ server: "Mã sinh viên này đã được đăng ký".
13	Chuyển sang trang đăng nhập	Nhấn vào liên kết "Đăng nhập" ở dưới form.	Chuyển hướng thành công sang trang Login.

14	Quay lại trang chủ	Nhấn liên kết Trang chủ.	Chuyển hướng về trang chủ thành công.
----	--------------------	--------------------------	---------------------------------------

b. Test Case chi tiết

- TC1: Kiểm tra giao diện đăng ký

```
[Test]
0 references
public void Test00_Register_UI_Display()
{
    driver.Navigate().GoToUrl("http://127.0.0.1:5000/register");
    driver.Manage().Window.Maximize();

    var wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));

    Assert.That(wait.Until(d => d.FindElement(By.Id("username"))), Is.Not.Null);
    Assert.That(wait.Until(d => d.FindElement(By.Id("email"))), Is.Not.Null);
    Assert.That(wait.Until(d => d.FindElement(By.Id("password"))), Is.Not.Null);
    Assert.That(wait.Until(d => d.FindElement(By.Id("student_code"))), Is.Not.Null);
    Assert.That(wait.Until(d => d.FindElement(By.Id("full_name"))), Is.Not.Null);
    Assert.That(wait.Until(d => d.FindElement(By.CssSelector(".btn-register"))), Is.Not.Null);
}
```

- TC2: Đăng ký thành công

```
[Test]
0 references
public void Test01_Register_Success()
{
    driver!.Navigate().GoToUrl(registerUrl);
    string randomId = DateTime.Now.Ticks.ToString().Substring(10);
    string user = "sv" + randomId;
    RegisterAction(user, user + "@neu.edu.vn", "Pass123456", "1123" + randomId.Substring(0, 4), "Sinh Viên Test");

    Thread.Sleep(2000);
    Assert.That(driver.Url, Does.Contain("login"));
}
```

- TC3: Để trống tên đăng nhập, email, mật khẩu, msv, họ tên

```

[Test]
[TestCase("", "a@neu.edu.vn", "123456", "1122", "Name", TestName = "Test02_Empty_Username")]
[TestCase("user", "", "123456", "1122", "Name", TestName = "Test03_Empty_Email")]
[TestCase("user", "a@neu.edu.vn", "", "1122", "Name", TestName = "Test04_Empty_Password")]
[TestCase("user", "a@neu.edu.vn", "123456", "", "Name", TestName = "Test05_Empty_StudentCode")]
[TestCase("user", "a@neu.edu.vn", "123456", "1122", "", TestName = "Test06_Empty_FullName")]
0 references
public void TestGroup_RequiredFields(string u, string e, string p, string m, string n)
{
    driver!.Navigate().GoToUrl(registerUrl);
    RegisterAction(u, e, p, m, n);

    Thread.Sleep(1500);
    Assert.That(driver.Url, Does.Contain("register"), $"Thất bại tại: {TestContext.CurrentContext.Test.Name}");
}

```

- TC4: Email sai định dạng

```
[Test]
0 references
public void Test07_Invalid_Email_Domain()
{
    driver!.Navigate().GoToUrl(registerUrl);
    RegisterAction("user7", "test@gmail.com", "Pass123456", "11223344", "Nguyễn Văn A");
    VerifyRegisterError("Email phải có đuôi @neu.edu.vn");
}
```

- TC5: Email thiếu phần định danh

```
[Test]
0 references
public void Test08_Email_Missing_Prefix()
{
    driver!.Navigate().GoToUrl(registerUrl);
    RegisterAction("user8", "@neu.edu.vn", "Pass123456", "11223345", "Nguyễn Văn B");
    Thread.Sleep(2000);
    // Trình duyệt chặn với thông báo "Please enter a part followed by '@'"
    Assert.That(driver.Url, Does.Contain("register"));
}
```

- TC6: Mật khẩu ít hơn ký tự tối thiểu

```
[Test]
0 references
public void Test09_Password_Too_Short()
{
    driver!.Navigate().GoToUrl(registerUrl);
    RegisterAction("user9", "test9@neu.edu.vn", "123", "11223346", "Nguyễn Văn C");
    VerifyRegisterError("Mật khẩu phải có ít nhất 6 ký tự");
}
```

- TC7: Trùng tên đăng nhập

```
[Test]
0 references
public void Test10_Duplicate_Username()
{
    driver!.Navigate().GoToUrl(registerUrl);
    RegisterAction("admin", "admin_new@neu.edu.vn", "Pass123456", "88889999", "Admin Fake");
    VerifyRegisterError("Tên đăng nhập này đã tồn tại");
}
```

- TC8: Trùng mã sinh viên

```
[Test]
0 references
public void Test11_Duplicate_StudentCode()
{
    driver!.Navigate().GoToUrl(registerUrl);
    RegisterAction("usertest11", "test11@neu.edu.vn", "Pass123456", "11236033", "Trang Test");
    VerifyRegisterError("Mã sinh viên này đã được đăng ký");
}
```

- TC9: Chuyển sang trang đăng nhập

```
[Test]
0 references
public void Test12_Link_To_Login()
{
    driver!.Navigate().GoToUrl(registerUrl);
    IJavaScriptExecutor js = (IJavaScriptExecutor)driver;
    var link = driver.FindElement(By.XPath("//a[contains(text(), 'Đăng nhập')]"));
    js.ExecuteScript("arguments[0].scrollIntoView(true);", link);
    Thread.Sleep(1000);
    js.ExecuteScript("arguments[0].click();", link);
    Thread.Sleep(2000);
    Assert.That(driver.Url, Does.Contain("login"));
}
```

- TC10: Quay lại trang chủ

```
[Test]
0 references
public void Test13_Back_To_Home()
{
    driver!.Navigate().GoToUrl(registerUrl);
    Thread.Sleep(1000);
    var link = driver.FindElement(By.CssSelector("a[href='/']"));
    ((IJavaScriptExecutor)driver).ExecuteScript("arguments[0].click();", link);
    Thread.Sleep(2000);
    Assert.That(driver.Url, Is.Not.EqualTo(registerUrl));
}
```

2.2. Đăng nhập tài khoản (Sinh viên)

a. Kịch bản kiểm thử

STT	Mô tả kịch bản	Các bước kiểm thử	Kết quả mong đợi
-----	----------------	-------------------	------------------

1	Kiểm tra giao diện đăng nhập	Mở trang đăng nhập và kiểm tra các thành phần giao diện	Hiển thị đúng theo thiết kế
2	Đăng nhập thành công	1. Truy cập trang đăng nhập. 2. Nhập Username: admin. 3. Nhập Password: admin123. 4. Nhấn nút 'Đăng nhập'.	Hệ thống đăng nhập thành công và chuyển hướng sang trang Dashboard.
3	Nhập sai tên đăng nhập	1. Nhập Username không tồn tại (user_khong_ton_tai). 2. Nhập Password đúng. 3. Nhấn 'Đăng nhập'.	Hiển thị thông báo 'Tên đăng nhập hoặc tài khoản không chính xác'
4	Nhập sai mật khẩu	1. Nhập đúng tên đăng nhập 2. Nhập mật khẩu sai 3. Nhấn 'Đăng nhập'	Hiển thị thông báo 'Tên đăng nhập hoặc tài khoản không chính xác'
5	Nhập thiếu tên đăng nhập	1. Để trống ô Username. 2. Nhập Password hợp lệ. 3. Nhấn 'Đăng nhập'.	Trình duyệt hiển thị cảnh báo: "Please fill out this field" tại ô trống
6	Nhập thiếu mật khẩu	1. Nhập Username, để trống Password. 2. Nhấn 'Đăng nhập'.	Trình duyệt hiển thị cảnh báo: "Please fill out this field" tại ô trống
7	Nhập thiếu cả tên đăng nhập và mật	1. Để trống cả hai ô.	Trình duyệt hiển thị cảnh báo: "Please fill

	khẩu	2. Nhấn 'Đăng nhập'.	out this field" tại ô trống
8	Chuyển sang trang đăng ký	1. Nhấn vào liên kết "Đăng ký ngay".	Chuyển hướng thành công sang giao diện Register.
9	Quay lại trang chủ	Nhấn vào liên kết quay về.	Chuyển hướng người dùng về lại Trang chủ.

b. Test Case chi tiết

- TC1: Kiểm tra giao diện đăng nhập

```

}
[Test]
0 references
public void Test00_UI_Display()
{
    driver!.Navigate().GoToUrl(loginUrl);
    var wait = new WebDriverWait(driver, TimeSpan.FromSeconds(15));
    Assert.Multiple(() =>
    {
        Assert.That(wait.Until(d => d.FindElement(By.Name("username"))).Displayed);
        Assert.That(wait.Until(d => d.FindElement(By.Name("password"))).Displayed);
        Assert.That(wait.Until(d => d.FindElement(By.CssSelector("button[type='submit']"))).Displayed);
    });
}

```

- TC2: Đăng nhập thành công

```

[Test]
0 references
public void Test01_Login_Success()
{
    driver!.Navigate().GoToUrl(loginUrl);
    LoginAction("admin", "admin123");

    var wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));
    // Chờ đến khi chuyển trang (URL không còn chứa /login)
    wait.Until(d => !d.Url.ToLower().Contains("/login"));
    Assert.That(driver.Url.ToLower(), Does.Not.Contain("login"));
}

```


- TC3: Nhập sai tên đăng nhập

```
[Test]
0 references
public void Test05_Login_Wrong_User()
{
    driver!.Navigate().GoToUrl(loginUrl);
    LoginAction("user_khong_ton_tai", "123456");
    VerifyLoginError("Tên đăng nhập hoặc mật khẩu không chính xác");
}
```

- TC4: Nhập sai mật khẩu

```
[Test]
0 references
public void Test06_Login_Wrong_Pass()
{
    driver!.Navigate().GoToUrl(loginUrl);
    LoginAction("admin", "sai_mat_khau_123");
    VerifyLoginError("Tên đăng nhập hoặc mật khẩu không chính xác");
}
```

- TC5: Để trống tên đăng nhập, mật khẩu

```
[Test]
[TestCase("", "admin123", TestName = "Test02_Login_Empty_Username")]
[TestCase("admin", "", TestName = "Test03_Login_Empty_Password")]
[TestCase("", "", TestName = "Test04_Login_Empty_All")]
0 references
public void TestGroup_Login_RequiredFields(string u, string p)
{
    driver!.Navigate().GoToUrl(loginUrl);
    LoginAction(u, p);

    Thread.Sleep(1500);
    // Trình duyệt chặn nên URL vẫn là trang login
    Assert.That(driver.Url.ToLower(), Does.Contain("login"));
}
```

- TC6: Chuyển sang trang đăng ký

```
[Test]
0 references
public void Test07_Link_To_Register()
{
    driver!.Navigate().GoToUrl(loginUrl);
    var wait = new WebDriverWait(driver!, TimeSpan.FromSeconds(10));
    var link = wait.Until(ExpectedConditions.ElementExists(By.CssSelector("a[href*='register']")));

    IJavaScriptExecutor js = (IJavaScriptExecutor)driver;
    js.ExecuteScript("arguments[0].scrollIntoView(true);", link);
    Thread.Sleep(500);
    js.ExecuteScript("arguments[0].click();", link);

    wait.Until(d => d.Url.ToLower().Contains("register"));
    Assert.That(driver.Url.ToLower(), Does.Contain("register"));
}
```

- TC7: Quay lại trang chủ

```
[Test]
0 references
public void Test08_Back_To_Home()
{
    driver!.Navigate().GoToUrl(loginUrl);
    var homeLink = driver!.FindElement(By.CssSelector("a[href='/']"));
    ((IJavaScriptExecutor)driver).ExecuteScript("arguments[0].click();", homeLink);

    Thread.Sleep(1500);
    Assert.That(driver.Url.ToLower(), Is.Not.EqualTo(loginUrl));
}
```

2.3. Đăng nhập tài khoản (CLB) (giống Sinh viên)

2.4. Đăng ký thành lập CLB (Sinh viên)

a. Kịch bản kiểm thử

STT	Mô tả kịch bản	Các bước kiểm thử	Kết quả mong đợi
1	Kiểm tra giao diện đăng ký CLB	Mở trang và kiểm tra các thành phần giao diện	Hiển thị đúng theo thiết kế
2	Đăng ký thành công	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Nhập đầy đủ thông tin: Tên	Hệ thống gửi đơn thành công và chuyển hướng về trang Student

		CLB mới, Lĩnh vực, Username mới, Mô tả. 4. Nhấn nút đăng ký.	Dashboard.
3	Điền trống tên Câu lạc bộ	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Điền trống ô tên Câu lạc bộ. 4. Nhập các trường còn lại. 5. Nhấn đăng ký.	Trình duyệt chặn lại và hiển thị thông báo "Please fill out this field."
4	Chưa chọn lĩnh vực	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Không chọn giá trị trong danh sách 'Lĩnh vực'. 4. Nhấn đăng ký.	Trình duyệt yêu cầu người dùng phải chọn một mục trong danh sách select.
5	Điền trống tên đăng nhập dự kiến	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Điền trống ô Username CLB. 4. Nhấn đăng ký.	Xuất hiện thông báo yêu cầu nhập liệu tại ô Tên đăng nhập dự kiến..
6	Điền trống mô tả hoạt động	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Điền trống ô Mô tả/Mục đích. 4. Nhấn đăng ký.	Trình duyệt yêu cầu hoàn thiện nội dung mô tả mục đích & hoạt động..

7	Trùng tên đăng nhập CLB	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Nhập Username CLB đã tồn tại 4. Nhấn đăng ký.	Hệ thống phản hồi lỗi từ phía máy chủ thông báo tên đăng nhập đã tồn tại.
8	Trùng tên Câu lạc bộ	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Nhập tên CLB đã có trong hệ thống . 4. Nhấn đăng ký.	Hệ thống phản hồi lỗi tên Câu lạc bộ đã được đăng ký trong cơ sở dữ liệu.
9	Kiểm tra tính năng Hủy	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Nhấn vào liên kết có nội dung "Hủy bỏ".	Hệ thống không gửi dữ liệu và chuyển hướng ngay lập tức về trang Student Dashboard.
10	Bỏ trống các trường tùy chọn	1. Đăng nhập 2. Vào trang Đăng ký CLB 3. Nhập đủ các trường bắt buộc. 4. Để trống SĐT/Email CLB. 5. Nhấn đăng ký.	Đăng ký thành công và chuyển hướng về trang Dashboard.

b. Test Case chi tiết

- TC1: Kiểm tra giao diện đăng ký clb

```
[Test]
0 references
public void Test_00_ClubRegistration_UI_Display()
{
    d!.Navigate().GoToUrl(url + "/student/club/register");

    Assert.That(w!.Until(driver => driver.FindElement(By.Name("club_name"))), Is.Not.Null);
    Assert.That(w!.Until(driver => driver.FindElement(By.Name("field"))), Is.Not.Null);
    Assert.That(w!.Until(driver => driver.FindElement(By.Name("intended_username"))), Is.Not.Null);
    Assert.That(w!.Until(driver => driver.FindElement(By.Name("email"))), Is.Not.Null);
    Assert.That(w!.Until(driver => driver.FindElement(By.Name("phone"))), Is.Not.Null);
    Assert.That(w!.Until(driver => driver.FindElement(By.Name("president_name"))), Is.Not.Null);
    Assert.That(w!.Until(driver => driver.FindElement(By.Name("description"))), Is.Not.Null);
    Assert.That(w!.Until(driver => driver.FindElement(By.CssSelector("button[type='submit']"))), Is.Not.Null);
}
```

- TC2: Đăng ký thành công

```
[Test]
0 references
public void Test_01_RegSuccess()
{
    string id = DateTime.Now.Ticks.ToString().Substring(10, 5);

    RunAction(
        "CLB Thành Công " + id,
        "Học thuật",
        "leader" + id,
        "Mô tả hoạt động CLB đầy đủ",
        "club" + id + "@neu.edu.vn",
        "09123" + id
    );

    var wait = new WebDriverWait(d!, TimeSpan.FromSeconds(15));
    wait.Until(driver => driver.Url.ToLower().Contains("dashboard"));

    Assert.That(d!.Url.ToLower(), Does.Contain("dashboard"));
}
```

- TC3: Bỏ trống các trường bắt buộc

```
[Test]
[TestCase("", "Học thuật", "u2", "Goal 02", TestName = "Test_02_EmptyName")]
[TestCase("C03", "", "u3", "Goal 03", TestName = "Test_03_EmptyField")]
[TestCase("C04", "Thể thao", "", "Goal 04", TestName = "Test_04_EmptyUser")]
[TestCase("C05", "Văn nghệ/Nghệ thuật", "u5", "", TestName = "Test_05_EmptyDesc")]
0 references
public void TestGroup_EmptyFields(string n, string f, string u, string m)
{
    RunAction(n, f, u, m, "", "");
    Thread.Sleep(1500);
    Assert.That(d!.Url.ToLower(), Does.Contain("register"));
}
```

- TC4: Trùng tên đăng nhập CLB

```
[Test]
0 references
public void Test_06_DupUser()
{
    RunAction("C06", "Học thuật", "Thư", "Goal 06", "", "");
    Assert.That(d!.Url.ToLower(), Does.Contain("register"));
}
```

- TC5: Trùng tên Câu lạc bộ

```
[Test]
0 references
public void Test_07_DupClub()
{
    RunAction("CLB Am Nhạc", "Học thuật", "u7", "Goal 07", "", "");
    Assert.That(d!.Url.ToLower(), Does.Contain("register"));
}
```

- TC6: Kiểm tra tính năng Hủy

```
[Test]
0 references
public void Test_08_Cancel()
{
    d!.Navigate().GoToUrl(url + "/student/club/register");
    var btnCancel = w!.Until(ExpectedConditions.ElementExists(By.LinkText("Hủy bỏ")));
    ((IJavaScriptExecutor)d).ExecuteScript("arguments[0].click();", btnCancel);
    Thread.Sleep(2000);
    Assert.That(d.Url.ToLower(), Does.Contain("dashboard"));
}
```

- TC7: Bỏ trống các trường tùy chọn

```
[Test]
0 references
public void Test_09_NoOpt()
{
    string id = DateTime.Now.Ticks.ToString().Substring(10, 5);
    RunAction("C09 " + id, "Học thuật", "u9_" + id, "Goal 09", "", "");
    Assert.That(d!.Url.ToLower(), Does.Contain("dashboard"));
}
```

2.5. Đăng ký tham gia CLB (Sinh viên)

a. Kịch bản kiểm thử

STT	Mô tả kịch bản	Các bước kiểm thử	Kết quả mong đợi
1	Kiểm tra giao diện đăng ký tham gia CLB	Mở trang và kiểm tra các thành phần giao diện	Hiển thị đúng theo thiết kế
2	Nộp đơn đăng ký tham gia CLB thành công	- Đăng nhập - Vào trang Khám phá CLB - Click + vào một CLB - Nhập lý do tham gia CLB - Gửi đơn ngay	Hiển thị thông báo “Nộp đơn thành công”
3	Nộp đơn thất bại từ trang danh	- Đăng nhập - Vào trang Khám phá CLB	Xuất hiện thông báo yêu cầu nhập liệu tại

	sách do bỏ trống lý do	3. Click + vào một CLB 4. Để trống lý do tham gia CLB 5. Gửi đơn ngay	ô Tại sao bạn tham gia?
4	Kiểm tra nộp đơn thành công từ trang chi tiết	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Xem thông tin chi tiết CLB 4. Chọn gửi đơn tham gia 5. Nhập lý do tham gia CLB 6. Gửi đơn ngay	Hiển thị thông báo “Nộp đơn thành công”
5	Kiểm tra nộp đơn thất bại do bỏ trống lý do (Trang chi tiết)	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Xem thông tin chi tiết 4. Chọn gửi đơn tham gia 5. Để trống lý do tham gia CLB 6. Gửi đơn ngay	Xuất hiện thông báo yêu cầu nhập liệu tại ô Tại sao bạn tham gia?
6	Nộp đơn đăng ký tham gia CLB từ trang danh sách CLB với CLB đã nộp đơn rồi	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Click + vào một CLB 4. Nhập lý do tham gia CLB 5. Gửi đơn ngay	Hiển thị thông báo đã nộp đơn tham gia CLB này. Hãy chờ duyệt
7	Kiểm tra nộp đơn vào CLB đã nộp đơn rồi (Trang chi tiết)	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Xem thông tin chi tiết 4. Chọn gửi đơn tham gia 5. Nhập lý do tham gia CLB 6. Gửi đơn ngay	Hiển thị thông báo đã nộp đơn tham gia CLB này. Hãy chờ duyệt
8	Kiểm tra nộp đơn vào CLB do mình sáng lập	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Click + vào một CLB	Hiển thị thông báo bạn là người sáng lập, không thể đăng

		4. Nhập lý do tham gia CLB 5. Gửi đơn ngay	ký.
9	Kiểm tra huy hiệu "Thành viên/Sáng lập" (Trang chi tiết)	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Xem thông tin chi tiết	Hiển thị huy hiệu đã là thành viên
10	Kiểm tra thông báo đơn bị từ chối (Trang Notifications)	1. Đăng nhập 2. Click vào Notifications	Hiển thị đơn gia nhập CLB đã bị từ chối
11	Kiểm tra thông báo đơn được duyệt (Trang Notifications)	1. Đăng nhập 2. Click vào Notifications	Hiển thị thông báo đã được duyệt vào CLB
12	Kiểm tra nộp đơn khi đã là thành viên	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Xem thông tin chi tiết 4. Chọn gửi đơn tham gia 5. Nhập lý do tham gia CLB 6. Gửi đơn ngay	Hiển thị thông báo đã là thành viên CLB
13	Kiểm tra nộp đơn khi đã bị từ chối	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Click + vào một CLB đã bị từ chối 4. Nhập lý do tham gia CLB 5. Gửi đơn ngay	Hiển thị thông báo đã đơn gia nhập bị từ chối
14	Kiểm tra nộp đơn khi đã bị từ chối (Trang chi tiết)	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Xem thông tin chi tiết 4. Chọn gửi đơn tham gia	Hiển thị thông báo đã đơn gia nhập bị từ chối

		5. Nhập lý do tham gia CLB 6. Gửi đơn ngay	
15	Kiểm tra chức năng tìm kiếm-có kết quả	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Nhập từ khóa vào ô tìm kiếm	Tìm thấy ít nhất một CLB chứa đúng tên
16	Kiểm tra chức năng tìm kiếm-không có kết quả	1. Đăng nhập 2. Vào trang Khám phá CLB 3. Nhập từ khóa vào ô tìm kiếm	Không hiển thị kết quả tìm kiếm

b. Test Case chi tiết

- TC1: Kiểm tra giao diện trang khám phá CLB

```
[Test]
0 references
public void TC0_ClubsPage_UI_Check()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    Assert.That(driver.Title, Does.Contain("Hệ thống"), "Title trình duyệt sai!");
    var header = wait.Until(ExpectedConditions.ElementIsVisible(By.CssSelector("h2"))).Text;
    Assert.That(header, Does.Contain("Khám Phá Cộng Đồng"), "Tiêu đề trang hiển thị sai!");
    var subtext = driver.FindElement(By.CssSelector(".text-muted")).Text;
    Assert.That(subtext, Does.Contain("Tìm kiếm và gia nhập"), "Mô tả trang hiển thị sai!");
    var searchInput = driver.FindElement(By.Id("searchInput"));
    Assert.That(searchInput.Displayed, Is.True, "Thanh tìm kiếm không hiển thị!");
    Assert.That(searchInput.GetAttribute("placeholder"), Does.Contain("Tìm tên câu lạc bộ"), "Placeholder sai!");
    var clubCards = driver.FindElements(By.CssSelector(".club-item"));
    Assert.That(clubCards.Count, Is.GreaterThan(0), "Không có CLB nào được hiển thị trên trang!");
    var firstCard = clubCards[0];
    Assert.Multiple(() =>
    {
        Assert.That(firstCard.FindElement(By.CssSelector("a.btn-primary-light")).Text, Does.Contain("Thông Tin Chi Tiết"));
        Assert.That(firstCard.FindElement(By.CssSelector("button[data-bs-target='#applyModal']")).Displayed, Is.True, "Nút '+' (Gửi đơn) không hiển thị!");
    });
}
```

- TC2: Nộp đơn thành công từ trang danh sách, nhập đủ lý do

```
[Test]
0 references
public void TC1_Apply_FromClubsPage_Success()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    string modalId = OpenModalByName("Test Club 176");

    FillMotivationByJS(modalId, "TC1: Em muốn tham gia CLB.");
    SubmitModalByJS(modalId, "Gửi Đơn Ngay");
    AssertFlashMessage(".alert-success", "Nộp đơn thành công");
}
```

- TC3: Nộp đơn thất bại từ trang danh sách do bỏ trống lý do

```
[Test]
| 0 references
public void TC2_Apply_FromClubsPage_MissingMotivation()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    string modalId = OpenModalById(4);
    SubmitModalByJS(modalId, "Gửi Đơn Ngay");
    var field = driver.FindElement(By.Id(modalId)).FindElement(By.Name("motivation"));
    Assert.That(field.GetAttribute("validationMessage"), Is.Not.Null.And.Not.Empty);
    Thread.Sleep(slowDelay);
}
```

- TC4: Kiểm tra nộp đơn THÀNH CÔNG từ trang CHI TIẾT (Club detail), vào xem thông tin CLB rồi mới quyết định nhấn nộp đơn.

```
[Test]
| 0 references
public void TC3_Apply_FromDetailPage_Success()
{
    driver.Navigate().GoToUrl(baseUrl + "/clubs/1");
    OpenDetailModal();
    FillMotivationByJS("joinClubModal", "TC3: Em rất thích các hoạt động của CLB.");
    SubmitModalByJS("joinClubModal", "Nộp Đơn Ứng Tuyển");
    AssertFlashMessage(".alert-success", "Nộp đơn thành công");
}
```

- TC5: Kiểm tra nộp đơn THẤT BẠI do BỎ TRỐNG lý do (Trang chi tiết)

```
[Test]
| 0 references
public void TC4_Apply_FromDetailPage_MissingMotivation()
{
    driver.Navigate().GoToUrl(baseUrl + "/clubs/1");
    OpenDetailModal();
    SubmitModalByJS("joinClubModal", "Nộp Đơn Ứng Tuyển");
    var field = driver.FindElement(By.Id("joinClubModal")).FindElement(By.Name("motivation"));
    Assert.That(field.GetAttribute("validationMessage"), Is.Not.Null.And.Not.Empty);
    Thread.Sleep(slowDelay);
}
```

- TC6: Kiểm tra nộp đơn vào CLB ĐÃ NỘP RỒI (Trang danh sách)

```
[Test]
✓ | 0 references
public void TC5_Apply_FromClubsPage_AlreadyApplied()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    string modalId = OpenModalByName("Test Club 176");
    FillMotivationByJS(modalId, "Tạo đơn lần 1");
    SubmitModalByJS(modalId, "Gửi Đơn Ngay");
    Thread.Sleep(1000);
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    OpenModalByName("Test Club 176");
    FillMotivationByJS(modalId, "Thử nộp lần 2");
    SubmitModalByJS(modalId, "Gửi Đơn Ngay");
    AssertFlashMessage(".alert-warning", "đang chờ duyệt");
}
```

- TC7: Kiểm tra nộp đơn vào CLB ĐÃ NỘP RỒI (Trang chi tiết)

```
[Test]
✓ | 0 references
public void TC6_Apply_AlreadyApplied_FromDetailPage()
{
    driver.Navigate().GoToUrl(baseUrl + "/clubs/1");
    OpenDetailModal();
    FillMotivationByJS("joinClubModal", "Tạo đơn lần 1");
    SubmitModalByJS("joinClubModal", "Nộp Đơn Ứng Tuyển");
    Thread.Sleep(1000);
    driver.Navigate().GoToUrl(baseUrl + "/clubs/1");
    OpenDetailModal();
    FillMotivationByJS("joinClubModal", "Thử nộp lần 2");
    SubmitModalByJS("joinClubModal", "Nộp Đơn Ứng Tuyển");
    AssertFlashMessage(".alert-warning", "đang chờ duyệt");
}
```

- TC8: Kiểm tra nộp đơn vào CLB DO MÌNH SÁNG LẬP (Founder)

```
[Test]
| 0 references
public void TC7_Apply_AsFounder_FromClubsPage()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    string modalId = OpenModalById(5);
    FillMotivationByJS(modalId, "Tôi là founder.");
    SubmitModalByJS(modalId, "Gửi Đơn Ngay");
    AssertFlashMessage(".alert-warning", "người sáng lập");
}
```

- TC9: Kiểm tra huy hiệu "Thành viên/Sáng lập" (Trang chi tiết)

```
[Test]
| 0 references
public void TC8_View_AlreadyMember_OnDetailPage()
{
    driver.Navigate().GoToUrl(baseUrl + "/clubs/5");
    Thread.Sleep(slowDelay);
    var container = wait.Until(ExpectedConditions.ElementIsVisible(By.CssSelector("div.mt-3.mt-md-0")));
    var btn = container.FindElement(By.CssSelector(".btn, .badge"));

    Assert.That(btn.Text, Does.Contain("Thành viên").Or.Contain("Sáng Lập").Or.Contain("Gửi Đơn"));
    Thread.Sleep(slowDelay);
}
```

- TC10: Kiểm tra thông báo đơn bị TỪ CHỐI (Trang Notifications)

```
[Test]
| 0 references
public void TC9_View_Rejected_Notification()
{
    driver.Navigate().GoToUrl(baseUrl + "/notifications");
    Thread.Sleep(slowDelay);
    var items = wait.Until(ExpectedConditions.VisibilityOfAllElementsLocatedBy(By.CssSelector(".list-group-item")));
    bool found = items.Any(n => n.Text.Contains("bị từ chối"));
    Assert.That(found, Is.True, "Không tìm thấy thông báo từ chối.");
}
```

- TC11: Kiểm tra thông báo đơn được DUYỆT (Trang Notifications)

```
[Test]
| 0 references
public void TC10_View_Approved_Notification()
{
    driver.Navigate().GoToUrl(baseUrl + "/notifications");
    Thread.Sleep(slowDelay);
    var items = wait.Until(ExpectedConditions.VisibilityOfAllElementsLocatedBy(By.CssSelector(".list-group-item")));
    bool found = items.Any(n => n.Text.Contains("được duyệt"));
    Assert.That(found, Is.True, "Không tìm thấy thông báo được duyệt.");
}
```

- TC12: Kiểm tra nộp đơn khi ĐÃ LÀ THÀNH VIÊN

```
[Test]
| 0 references
public void TC11_Apply_AlreadyMember_Error()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    string modalId = OpenModalByName("CLB Talents");
    FillMotivationByJS(modalId, "Thử nộp lại khi đã là thành viên");
    SubmitModalByJS(modalId, "Gửi Đơn Ngay");
    AssertFlashMessage(".alert-info", "Bạn đã là thành viên của CLB này.");
}
```

- TC13: Kiểm tra nộp đơn khi ĐÃ BỊ TỪ CHỐI

```
[Test]
| 0 references
public void TC12_Apply_AlreadyRejected_Error()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    string modalId = OpenModalByName("CLB Talents");

    FillMotivationByJS(modalId, "Thử nộp lại khi đã bị từ chối");
    SubmitModalByJS(modalId, "Gửi Đơn Ngay");

    AssertFlashMessage(".alert-danger", "đã bị từ chối");
}
```

- TC14: Kiểm tra nộp đơn khi ĐÃ BỊ TỪ CHỐI (Vào từ trang chi tiết)

```
[Test]
| 0 references
public void TC13_AlreadyRejected_Error()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    var card = wait.Until(ExpectedConditions.ElementIsVisible(By.XPath("//h6[contains(., 'CLB Talents')]/ancestor::div[contains(@class, 'club-item')]"));
    var detailBtn = card.FindElement(By.CssSelector("a.btn-primary-light"));
    ((IJavaScriptExecutor)driver).ExecuteScript("arguments[0].click();", detailBtn);
    OpenDetailModal();
    FillMotivationByJS("joinClubModal", "Cố tình nộp lại từ trang chi tiết");
    SubmitModalByJS("joinClubModal", "Nộp Đơn Ứng Tuyển");
    AssertFlashMessage(".alert-danger", "đã bị từ chối");
}
```

- TC15: Kiểm tra chức năng TÌM KIẾM - CÓ KẾT QUẢ

```
[Test]
0 references
public void TC14_Search_Found()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    var input = driver.FindElement(By.Id("searchInput"));

    input.Clear();
    Thread.Sleep(1000);
    string searchText = "Talents";
    foreach (char c in searchText)
    {
        ((IJavaScriptExecutor)driver).ExecuteScript(
            "arguments[0].value += arguments[1];" +
            "arguments[0].dispatchEvent(new Event('input', { bubbles: true }));",
            input, c.ToString());
        Thread.Sleep(200);
    }
    Thread.Sleep(3000);

    var isVisible = driver.FindElements(By.CssSelector(".club-item")).Any(c => c.Displayed && c.Text.Contains(searchText));
    Assert.That(isVisible, Is.True);
}
```

- TC16: Kiểm tra chức năng TÌM KIẾM - KHÔNG CÓ KẾT QUẢ

```
[Test]
0 references
public void TC15_Search_NotFound()
{
    driver.Navigate().GoToUrl(baseUrl + "/student/clubs");
    var input = driver.FindElement(By.Id("searchInput"));

    input.Clear();
    Thread.Sleep(1000);
    string searchText = "CLB_A0_9999";
    foreach (char c in searchText)
    {
        ((IJavaScriptExecutor)driver).ExecuteScript(
            "arguments[0].value += arguments[1];" +
            "arguments[0].dispatchEvent(new Event('input', { bubbles: true }));",
            input, c.ToString());
        Thread.Sleep(200);
    }
    Thread.Sleep(3000);

    var anyVisible = driver.FindElements(By.CssSelector(".club-item")).Any(c => c.Displayed);
    Assert.That(anyVisible, Is.False);
}
```

2.6. Phê duyệt đơn đăng ký tham gia CLB Sinh viên (CLB)

a. Kịch bản kiểm thử

STT	Mô tả kịch bản	Các bước kiểm thử	Kết quả mong đợi
1	Kiểm tra giao diện trang Duyệt đơn apply	- Đăng nhập với tài khoản CLB (talents) - Đến trang CLB	Hiển thị theo đúng thiết kế

2	CLB phê duyệt đơn tham gia CLB thành công	1. Sinh viên ("lan1") đăng nhập và gửi đơn vào CLB "Talents". 2. Đăng xuất tài khoản sinh viên. 3. Chủ nhiệm CLB ("talents") đăng nhập và nhấn nút "Duyệt" tại đơn của sinh viên.	- Hệ thống hiển thị thông báo thành công: "Đã duyệt đơn đăng ký tham gia". - Đơn ứng tuyển được cập nhật trạng thái thành công.
3	CLB từ chối đơn tham gia CLB nhưng bấm hủy xác nhận	1. Sinh viên ("lan2") đăng nhập và gửi đơn ứng tuyển. 2. Chủ nhiệm CLB đăng nhập, tìm đơn và nhấn nút "Từ chối". 3. Khi hộp thoại xác nhận hiện lên, nhấn nút "Hủy"	- Đơn ứng tuyển không bị thay đổi trạng thái. - Trạng thái của đơn vẫn hiển thị là "Đang chờ duyệt".
4	CLB từ chối đơn tham gia CLB thành công	1. Sinh viên ("lan3") đăng nhập và gửi đơn ứng tuyển. 2. Chủ nhiệm CLB đăng nhập, tìm đơn và nhấn nút "Từ chối". 3. Khi hộp thoại xác nhận hiện lên, nhấn nút "xác nhận"	- Hệ thống hiển thị thông báo: "Đã từ chối đơn đăng ký". - Trạng thái đơn ứng tuyển được cập nhật là đã từ chối.

b. Test Case chi tiết

- TC1: Kiểm tra giao diện duyệt đơn đăng ký tham gia CLB

```

[Test]
0 references
public void TC0_ClubApplications_UI_Check()
{
    Login("talents", "123456");
    driver.Navigate().GoToUrl(baseUrl + "/club/applications");
    Assert.That(driver.Title, Does.Contain("Hệ thống"), "Title sai!");
    var pageTitle = wait.Until(ExpectedConditions.ElementIsVisible(By.CssSelector(".page-title"))).Text;
    Assert.That(pageTitle, Does.Contain("Đơn Đăng Ký Gia Nhập"));
    Assert.That(driver.FindElement(By.TagName("thead")).Text, Does.Contain("Sinh Viên"));
}

```


- TC2: Quy trình duyệt đơn thành công

```
[Test]
0 references
public void TC1_StudentApply_ClubApprove_Success()
{
    Login("lan1", "123456");
    ApplyToClub("CLB Talents");
    Logout();
    Login("talents", "123456");
    ProcessFirstApplication("approve");
    var alert = wait.Until(ExpectedConditions.ElementIsVisible(By.CssSelector(".alert-success")));
    Assert.That(alert.Text, Does.Contain("Đã duyệt đơn đăng ký tham gia"));
}
```

- TC3: từ chối đơn nhưng bấm "hủy" (cancel) -> không thay đổi

```
[Test]
0 references
public void TC2_StudentApply_ClubReject_Cancel()
{
    Login("lan2", "123456");
    ApplyToClub("CLB Talents");
    Logout();
    Login("talents", "123456");
    driver.Navigate().GoToUrl(baseUrl + "/club/applications");
    Thread.Sleep(3000);
    var rejectBtn = wait.Until(ExpectedConditions.ElementToBeClickable(
        By.XPath("//button[contains(@class, 'btn-outline-danger') and contains(., 'Từ chối')]")));
    ((IJavaScriptExecutor)driver).ExecuteScript("arguments[0].click();", rejectBtn);
    Thread.Sleep(1000);
    wait.Until(ExpectedConditions.AlertIsPresent()).Dismiss();
    Thread.Sleep(3000);
    var statusBadge = driver.FindElement(By.CssSelector(".badge.bg-warning"));
    Assert.That(statusBadge.Text, Does.Contain("Đang chờ duyệt"));
}
```

- TC4: quy trình từ chối đơn thành công (bấm ok)

```
[Test]
0 references
public void TC3_StudentApply_ClubReject_Success()
{
    Login("lan3", "123456");
    ApplyToClub("CLB Talents");
    Logout();

    Login("talents", "123456");
    ProcessFirstApplication("reject");

    var alert = wait.Until(ExpectedConditions.ElementIsVisible(By.CssSelector(".alert")));
    Assert.That(alert.Text, Does.Contain("Đã từ chối đơn đăng ký"));
}
```

2.7. Phê duyệt đơn thành lập CLB

a. Kịch bản kiểm thử

STT	Mô tả kịch bản	Các bước kiểm thử	Kết quả mong đợi
1	Kiểm tra giao diện trang CLB	3. Đăng nhập với tài khoản admin 4. Đến trang CLB	Hiển thị theo đúng thiết kế
2	Admin phê duyệt đơn thành lập CLB thành công	4. Đăng nhập với tài khoản admin 5. Đến trang CLB 6. Tìm 1 CLB có trạng thái “Chờ duyệt” 7. Nhấn biểu tượng Duyệt	Hiển thị thông báo “Đã phê duyệt câu lạc bộ” + tên CLB
3	Admin từ chối đơn thành lập CLB thành công	4. Đăng nhập với tài khoản admin 5. Đến trang CLB 6. Tìm 1 CLB có trạng thái “Chờ duyệt” 7. Nhấn biểu tượng Từ chối	Hiển thị thông báo “Đã từ chối câu lạc bộ” + tên CLB
4	Tìm kiếm CLB - Có kết quả	4. Đăng nhập với tài khoản admin 5. Đến trang CLB 6. Nhập từ khóa tồn tại (ví dụ: "CLB Talents") vào ô tìm kiếm.	Hiển thị CLB Talents
5	Tìm kiếm CLB - Không kết quả	1. Đăng nhập với tài khoản admin 2. Đến trang CLB 3. Nhập từ khóa tồn tại (ví dụ: "CLB Talents") vào ô tìm kiếm.	Danh sách không hiển thị bất kỳ CLB nào

b. Test Case chi tiết

- TC1: Kiểm tra giao diện trang quản lý CLB

```
[Test]
0 references
public void Test_00_AdminClubs_UI_Check()
{
    Login("admin", "admin123");
    driver!.Navigate().GoToUrl(url + "/admin/clubs");
    Assert.That(driver.Title, Does.Contain("Hệ thống"), "Tiêu đề trình duyệt không chứa từ khóa mong muốn!");
    var headerElement = wait!.Until(ExpectedConditions.ElementIsVisible(By.TagName("h3")));
    Assert.That(headerElement.Text, Does.Contain("Quản Lý Câu Lạc Bộ"), "Nội dung Header không đúng!");
    var searchInput = driver.FindElement(By.Id("searchInput"));
    Assert.That(searchInput.Displayed, Is.True, "Ô tìm kiếm không hiển thị!");
}
```

- TC2: Admin phê duyệt đơn thành lập CLB thành công

```
[Test]
0 references
public void Test_01_ApproveClub_Success()
{
    Login("admin", "admin123");
    driver!.Navigate().GoToUrl(url + "/admin/clubs");
    var pendingRow = GetClubRowByStatus("Chờ duyệt");
    if (pendingRow == null) Assert.Ignore("Không tìm thấy CLB nào ở trạng thái 'Chờ duyệt'.");
    var approveBtn = pendingRow.FindElement(By.CssSelector("form input[value='approve'] + button"));
    ((IJavaScriptExecutor)driver!).ExecuteScript("arguments[0].click();", approveBtn);
    var alert = wait!.Until(ExpectedConditions.ElementIsVisible(By.ClassName("alert-success")));
    Assert.That(alert.Text, Does.Contain("Đã phê duyệt"), "Thông báo phê duyệt không hiển thị đúng!");
}
```

- TC3: Admin từ chối đơn thành lập CLB thành công

```
[Test]
0 references
public void Test_02_RejectClub_Success()
{
    Login("admin", "admin123");
    driver!.Navigate().GoToUrl(url + "/admin/clubs");
    var pendingRow = GetClubRowByStatus("Chờ duyệt");
    if (pendingRow == null) Assert.Ignore("Không tìm thấy CLB nào ở trạng thái 'Chờ duyệt'.");
    var rejectBtn = pendingRow.FindElement(By.CssSelector("form input[value='reject'] + button"));
    ((IJavaScriptExecutor)driver!).ExecuteScript("arguments[0].click();", rejectBtn);
    var alert = wait!.Until(ExpectedConditions.ElementIsVisible(By.ClassName("alert-warning")));
    Assert.That(alert.Text, Does.Contain("Đã từ chối"), "Thông báo từ chối không hiển thị đúng!");
}
```

- TC4: Tìm kiếm CLB - Có kết quả

```
[Test]
0 references
public void Test_03_Search_FoundResults()
{
    Login("admin", "admin123");
    driver!.Navigate().GoToUrl(url + "/admin/clubs");

    var searchInput = driver.FindElement(By.Id("searchInput"));
    string keyword = "CLB Talents";
    searchInput.Clear();
    foreach (char c in keyword) { searchInput.SendKeys(c.ToString()); Thread.Sleep(100); }

    Thread.Sleep(2000);
    var visibleRows = driver.FindElements(By.CssSelector(".club-row")).Where(r => r.Displayed).ToList();
    Assert.That(visibleRows.Count, Is.GreaterThan(0), "Tìm kiếm có kết quả nhưng không hiển thị hàng nào!");
}
```

- TC5: Tìm kiếm CLB - Không có kết quả

```
[Test]
0 references
public void Test_04_Search_NoResults()
{
    Login("admin", "admin123");
    driver!.Navigate().GoToUrl(url + "/admin/clubs");

    var searchInput = driver.FindElement(By.Id("searchInput"));
    searchInput.Clear();
    searchInput.SendKeys("Kiem_Tra_Khong_Ton_Tai_123");

    Thread.Sleep(2000);
    var visibleRows = driver.FindElements(By.CssSelector(".club-row")).Where(r => r.Displayed).ToList();
    Assert.That(visibleRows.Count, Is.EqualTo(0), "Vẫn hiển thị kết quả dù từ khóa không tồn tại!");
}
```

3. Báo cáo kết quả kiểm thử

Sau khi thực hành kiểm thử các chức năng chính, nhóm đưa ra được báo cáo kiểm thử như sau:

Tổng số test case: 54	Thành công	54
	Thất bại	0
	Lỗi	0
	Bỏ qua	0

The screenshot displays the Visual Studio Test Explorer interface. At the top, the 'Test Explorer' tab is active, showing a summary of the test run: 'Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 13 sec'. Below this, a table lists the test results:

Test	Duration	Traits	E.
✓ SeleniumTests (54)	12.5 min		
✓ SeleniumTests (54)	12.5 min		
✓ AdminClubManagementTests (5)	1 min		
✓ ApplyClubTests (16)	4.9 min		
✓ ClubRegistrationTests (6)	1.3 min		
✓ FullMembershipProcessTests (4)	1.9 min		
✓ LoginTests (9)	57.6 sec		
✓ RegisterTests (14)	2.4 min		

On the right side, the 'Group Summary' for 'SeleniumTests' is shown, indicating 'Tests in group: 54' and 'Total Duration: 12.5 min'. Below this, the 'Outcomes' section shows '54 Passed' with a green checkmark icon.

KẾT LUẬN

Trong khuôn khổ môn học *Kiểm thử và Đảm bảo chất lượng phần mềm*, đề tài đã tập trung nghiên cứu và ứng dụng kiểm thử tự động trên nền tảng .NET thông qua công cụ Selenium WebDriver. Việc lựa chọn Selenium kết hợp với ngôn ngữ C# trong môi trường .NET không chỉ phù hợp với xu hướng phát triển phần mềm hiện nay mà còn tạo điều kiện thuận lợi cho việc xây dựng các kịch bản kiểm thử có tính tổ chức và khả năng mở rộng cao.

Dựa trên việc phân tích hệ thống quản lý và kết nối Câu lạc bộ – sinh viên, nhóm đã xác định các chức năng trọng tâm và tiến hành xây dựng các kịch bản kiểm thử tương ứng. Quá trình thực hiện cho thấy kiểm thử tự động bằng .NET Selenium mang lại nhiều lợi ích rõ rệt, đặc biệt trong việc tự động hóa các thao tác lặp lại, rút ngắn thời gian kiểm thử và nâng cao độ chính xác so với phương pháp kiểm thử thủ công. Đồng thời, công cụ này cũng hỗ trợ hiệu quả cho kiểm thử hồi quy khi hệ thống có sự thay đổi hoặc nâng cấp.

Một điểm đáng chú ý trong quá trình triển khai là việc áp dụng mô hình Page Object Model nhằm tổ chức mã kiểm thử theo hướng tách biệt giữa logic kiểm thử và giao diện. Cách tiếp cận này giúp tăng khả năng tái sử dụng, giảm thiểu sự phụ thuộc vào thay đổi của giao diện người dùng và hỗ trợ tốt cho việc bảo trì lâu dài. Qua đó, nhóm nhận thức rõ hơn vai trò của việc thiết kế một framework kiểm thử hợp lý trong hoạt động đảm bảo chất lượng phần mềm.

Bên cạnh những kết quả đạt được, đề tài vẫn còn tồn tại một số hạn chế như phạm vi kiểm thử chưa bao phủ toàn bộ hệ thống, việc xử lý các phần tử động trên giao diện còn gặp khó khăn và chưa tích hợp kiểm thử vào quy trình phát triển liên tục (CI/CD). Tuy nhiên, những hạn chế này cũng mở ra hướng phát triển trong tương lai, chẳng hạn như mở rộng kiểm thử sang API, kiểm thử hiệu năng và hoàn thiện hệ thống kiểm thử tự động theo hướng toàn diện hơn.

Tóm lại, đề tài đã đạt được các mục tiêu đề ra, đồng thời giúp nhóm củng cố kiến thức lý thuyết và nâng cao kỹ năng thực hành trong lĩnh vực kiểm thử phần mềm. Việc ứng dụng .NET Selenium không chỉ mang lại hiệu quả trong kiểm thử mà còn góp phần khẳng định vai trò quan trọng của kiểm thử tự động trong việc đảm bảo chất lượng và độ tin cậy của các hệ thống phần mềm hiện đại.

Giảng viên có thể xem chi tiết web demo và code test tại đây:

<https://github.com/Hoaithu2501/SeleniumTests.git>

DANH SÁCH TÀI LIỆU THAM KHẢO

1. Selenium Documentation: hướng dẫn về WebDriver, các loại Wait (Explicit/Implicit) và cách tương tác với Element.
<https://www.selenium.dev/documentation/>
2. Microsoft .NET Documentation (C#): Tài liệu về ngôn ngữ C# và các thư viện hệ thống
<https://learn.microsoft.com/en-us/dotnet/csharp/>
3. NUnit Framework Guide: Hướng dẫn sử dụng các Annotation
<https://docs.nunit.org/>
4. Selenium.WebDriver.ChromeDriver: Trình điều khiển trình duyệt Chrome tự động.
<https://www.nuget.org/packages/Selenium.WebDriver.ChromeDriver/>
5. DotNetSeleniumExtras (WaitHelpers)
<https://www.nuget.org/packages/DotNetSeleniumExtras.WaitHelpers/>